

Staatsexamen: Informatik

Alle Themen und Aufgabentypen

**Jean-Baptiste Bellynck, Paul Kube,
Matthias Rips & Sabrina Zick**

0.1 Vorwort

Im folgenden ein paar Sätze zum Buch:

- An wen richtet sich das Buch?
- Was enthält das Buch, was enthält es nicht?
- Wie liest man dieses Buch?
- Wie sind die einzelnen Kapitel strukturiert?

Inhaltsverzeichnis

1 Theoretische Informatik

Programmieren ist ein zentraler Bestandteil der Informatik. Man versucht damit meist ein gewisses Problem zu lösen. Vielleicht will man ein fesselndes Spiel programmieren und muss dazu einen Gegner schreiben, die bestimmte Anforderungen erfüllt? Vielleicht will man eine Menge von Daten auswerten? Oder man will einfach den Wert einer mathematischen Funktion berechnen? Ein mathematisch geneigter Programmier könnte sich aber dann vor dem Programmieren einige wichtige Fragen stellen:

1. Kann ich das Problem überhaupt lösen?
2. Kann ich das Problem mit meiner Programmiersprache lösen?
3. Kann ich das Problem in angemessener Zeit lösen?

Die Antworten auf diese Fragen ist nicht intuitiv. So gibt es Probleme, die sind gar nicht lösbar . Auch die zweite Frage birgt einige Überraschungen: So würden die meisten Menschen behaupten, dass man mit Origami unmöglich zwei Matrizen multiplizieren kann, während es in Python ganz einfach ist. Allerdings lässt sich mit Origami nicht nur das Produkt zweier Matrizen berechnen; jeder Pythoncode lässt sich auch auf einem (unendlich großen) Origami durch bestimmte Faltungen simulieren . Und selbst wenn sich ein gewisses Problem durch Programmieren lösen lässt, kann es sein, dass die Lösung unangemessen lange braucht. Der schnellste bisher bekannte Algorithmus für das [Travelling Salesman Problem] mit [vielen] Städten würde auf aktuellen Supercomputern länger als die bisherige Lebenszeit des Universums benötigen. .

In diesen Teil geben wir erst einen Überblick über sehr einfache Klassen von formalen Sprachen. Wir fangen mit regulären Sprachen an und arbeiten uns zu immer komplexer werdenden Klassen von Sprachen hoch, bis wir bei richtigen Programmiersprachen ankommen.. Dann behandeln wir Konzepte, mit denen wir die oberen drei Fragen beantworten können.

Das müssen wir alles thematisieren:

- Schreibweisen bei Sprachen und Grammatiken
- Myhill-Nerode
- Chomsky-Hierarchie (frei, kontextfrei, regulär)
- Regex, CYK-Algorithmus
- Entscheidbarkeit (v.a was ist Gödelnummer, Gödelisierung einer mit w codierten Turingmaschine)
- P/NP Begriffsfeld
- Turingmaschinen
- Reduktionen

Beispiel
fehlt

Beleg

Daten
einfügen

Was hat
das alles
eigentlich
mit der
Schule zu
tun?

- Arten von Automaten (denk auch an Kellerautomat)
- Minimaler Automat (Minimierungstabelle), Potenzmengenkonstruktion, DFA Konstruktion

Notizen:

- Manche Begriffe haben verschiedene Terminologien und wir müssen alle erwähnen, falls die Wörter mal im Staatsexamen drankommen.
- Die einzelnen Subsections sollten dem gleichen System folgen.

1.1 Grundlagen

1.1.1 Grammatiken

Definition: Alphabet

Ein **Alphabet** Σ ist eine endliche nicht-leere Menge von **Zeichen** (oder Symbolen).

Beispiele:

- $\Sigma = \{a, b, c\}$
- $\Sigma = \{x, y\}$

Definition: Grammatik

Eine **Grammatik** ist ein 4-Tupel $G = (V, \Sigma, P, S)$ mit

- V ist eine endliche Menge von **Variablen** (alternativ Nichtterminale, Nichtterminalsymbole)
- Σ (mit $V \cap \Sigma = \emptyset$) ist ein **Alphabet** von **Zeichen** (alternativ Terminale, Terminalsymbole)
- P ist eine endliche Menge von **Produktionen** von der Form $l \rightarrow r$ wobei $l \in (V \cup \Sigma)^+$ und $r \in (V \cup \Sigma)^*$
- $S \in V$ ist das **Startsymbol** (alternativ Startvariable)

Definition: Notationen

- Eine Sprache L über einem Alphabet Σ , ist eine Teilmenge von Σ^* .
- L^* bezeichnet den **Kleenschen Abschluss**:

$$L^* := \bigcup_{i \geq 0} L^i$$

- L^+ ist der Kleensche Abschluss nur ohne dem leeren Wort.

Definition: Erzeugte Grammatik

Die von einer Grammatik $G = (V, \Sigma, P, S)$ ist die erzeugte Sprache $L(G)$ mit:

$$L(G) := \{w \in \Sigma^* : S \Rightarrow_G^* w\}$$

Erstmal sollte man eine Grammatik definieren.

1.1.2 Chomsky-Hierarchie

Die Grammatiken wurden von Chomsky in verschiedene Klassen unterteilt:

Definition: Chomsky-Hierarchie



Sei $G = (V, \Sigma, P, S)$ eine Grammatik. Seien $l, r \in \Sigma^*$ Wörter.

- Jede Grammatik ist **Typ 0**.
- G ist **kontextsensitiv/Typ 1**, wenn alle Produktionsregeln $l \rightarrow r$ folgende Ungleichung erfüllen:

$$|l| \leq |r|$$

d.h. das Wort links ist kleiner als das Wort rechts.

- G ist **kontextfrei/Typ 2**, wenn alle Produktionsregeln $l \rightarrow r$ von folgender Form sind:

$$A \rightarrow r$$

- G ist **regulär/rechtsregulär/rechtslinear/Typ 3**, wenn alle Produktionsregeln $l \rightarrow r$ von folgender Form sind:

$$A \rightarrow a, A \rightarrow B, A \rightarrow aB$$

- G ist **linksregulär/linkslinear**, wenn alle Produktionsregeln $l \rightarrow r$ von folgender Form sind:

$$A \rightarrow a, A \rightarrow B, A \rightarrow Ba$$

Wobei $a \in \Sigma$ Terminale und $A, B \in V$ Nicht-Terminale sind

Für Typ-2-Grammatiken gibt es immer eine Rechts- und Linksableitung. Das heißt, es wird entweder erst das linkeste oder rechteste nicht-Terminal abgeleitet.

Für alle diese Grammatiken gibt es die ϵ -Regel, die es erlaubt, dass das Startsymbol einmal in das leere Wort abgeleitet werden darf. Die restlichen Eigenschaften gehen dadurch nicht verloren.

Definition:

Eine Sprache L wird **kontextfrei/kontextsensitiv/regulär** genannt, wenn es eine Grammatik G mit $L = L(G)$ gibt, die **kontextfrei/kontextsensitiv/regulär** ist.

Nicht alle Bücher nutzen die gleiche Definition, besonders in Bezug auf die ϵ -Regel. Das muss beachtet werden.

1.1.3 Normalformen

1.1.4 Automaten und Maschinen

1.1.5 Entscheidbarkeit

Definition: Entscheidbarkeit

Eine Sprache heißt **entscheidbar/rekursiv**, wenn es einen Algorithmus gibt, der bei der Eingabe einer Grammatik G und einem Wort w in endlicher Zeit feststellt, ob $w \in L(G)$ oder nicht gilt.

Dabei sind alle Sprachen vom Typ 1,2 und 3 **entscheidbar**. Die Frage, ob ein Wort in einer Sprache enthalten ist, wird das **Wortproblem** genannt.

1.2 Reguläre Sprachen

1.2.1 Deterministische Endliche Automaten

Satz: Reguläre Sprachen in Automaten



M ist ein DFA $\iff L(M)$ regulär.

Für jede reguläre Sprache gibt es einen NFA, der sie akzeptiert (Rabin & Scott:

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.)

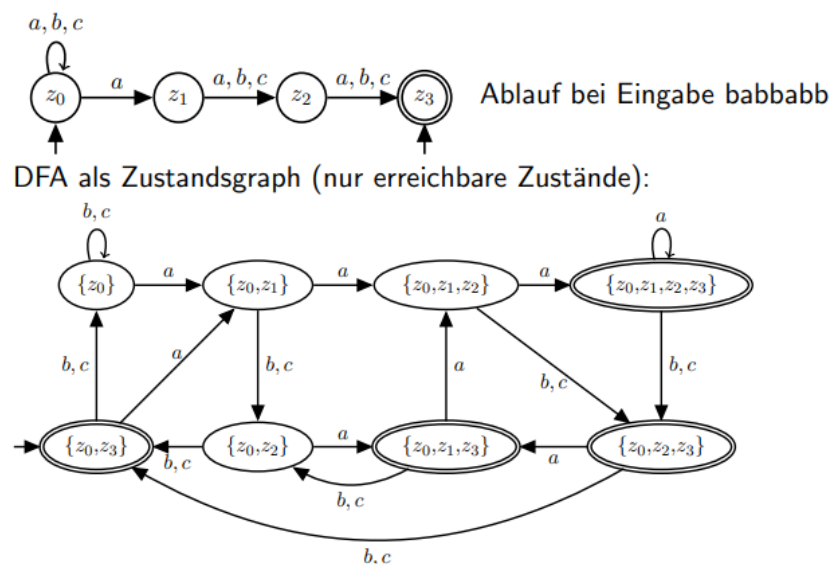
Ebenso akzeptieren NFAs mit ϵ -Übergängen genau die regulären Sprachen.

Potenzmengenkonstruktion

Über die Potenzmengenkonstruktion ist es möglich einen NFA in einen DFA zu überführen. Dabei sind:

- Die Zustände im DFA Mengen aus Zuständen des NFA.
- Jeder Zustand, der einen Endzustand des NFA enthält ist selber Endzustand.
- Die Startzustände werden in einem Startzustand gebündelt.

Vorgehen: Führe von jedem Knoten eine Verbindung zu einem Anderen, der genau alle erreichbaren Zustände enthält. Falls die Kombination noch nicht existiert, lege sie neu an und vervollständige zuerst alle nicht fertigen Knoten.



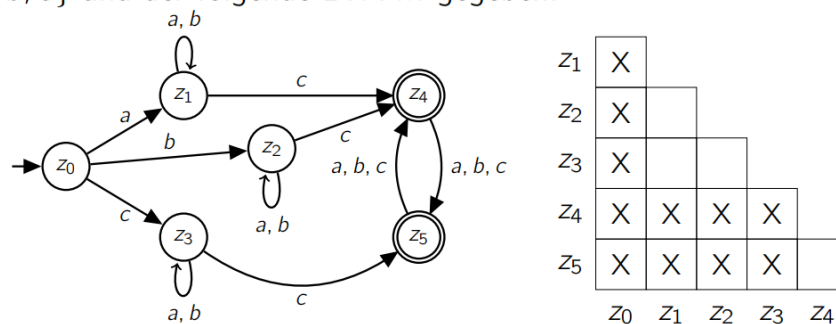
Minimierung eines DFAs

Hierbei geht es darum einen minimalen Automaten zu konstruieren. Dieser enthält dann nicht mehr Zustände als Knoten sondern Äquivalenzklassen von Zuständen.

Gehe dazu wie folgt vor.

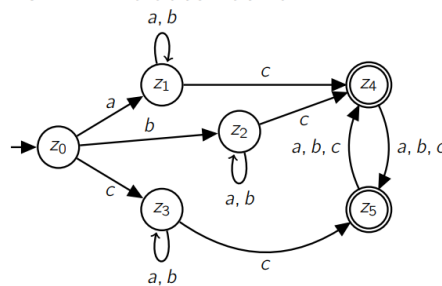
1. Entferne nicht erreichbare Zustände.
2. Zeichne die Tabelle wie im Beispielbild und markiere alle Tupel $\{z, z'\}$ für $z \notin E$ und $z' \in E$
3. Füge dann überall Markierungen zu Tupeln $\{z_i, z_j\}$ hinzu, für die gilt: $\exists a \in \Sigma : \{\delta(z_i, a), \delta(z_j, a)\}$ ist markiert.
4. Die leeren Felder sind Äquivalenzen von Zuständen.
5. Führe alle äquivalenten Zustände in eine Klasse zusammen und bilde aus den Klassen die neuen Knoten. Füge dann die Produktionen ein. Fertig.

Sei $\Sigma = \{a, b, c\}$ und der folgende DFA M gegeben:

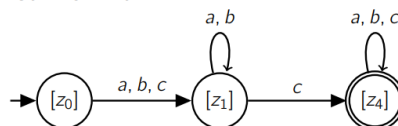


- ▶ alle Zustände sind von z_0 aus erreichbar
- ▶ äquivalente Zustände berechnen: Tabelle T erstellen
- ▶ Initiales Markieren: $\{z, z'\}$ mit $z \in \{z_0, z_1, z_2, z_3\}$ und $z' \in \{z_4, z_5\}$
- ▶ $\{z_0, z_1\}$, da $\{\delta(z_0, c), \delta(z_1, c)\} = \{z_3, z_4\}$ bereits markiert ist,
- ▶ $\{z_0, z_2\}$, da $\{\delta(z_0, c), \delta(z_2, c)\} = \{z_3, z_4\}$ bereits markiert ist, und
- ▶ $\{z_0, z_3\}$, da $\{\delta(z_0, c), \delta(z_3, c)\} = \{z_3, z_5\}$ bereits markiert

Der Minimalautomat zu



ist hiermit



Ich mag die Minimierungstabelle nicht, weil sie verschleiert, was wir hier machen. Dadurch ist der Inhalt in meinen Augen

1.2.2 Reguläre Ausdrücke

Definition: Reguläre Ausdrücke



Sei Σ ein Alphabet. Ein regulärer Ausdruck ist induktiv definiert:

- $\emptyset$ ist ein regulärer Ausdruck.
- ϵ ist ein regulärer Ausdruck.
- $a \in \Sigma$ ist ein regulärer Ausdruck.
- Sind a_1, a_2 reguläre Ausdrücke, dann sind es auch: a_1a_2 , $(a_1|a_2)$ und $(a_1)^*$.

Dabei ist $(a_1|a_2)$ die Auswahl.

Satz: Satz von Kleene



Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

1.2.3 Entscheidbarkeit

1.2.4 Charakterisierungen (Satz von Myhill-Nerode)

Der Satz von Myhill-Nerode ermöglicht eine genaue Definition der regulären Sprachen. Dafür benötigt man die Nerode-Relation:

Definition: Nerode-Relation

Sei L eine formale Sprache über Σ . Die Nerode-Relation ist definiert durch:

$$u \sim_L v \iff \forall w \in \Sigma^*: uw \in L \iff vw \in L$$

Satz: Myhill-Nerode

Eine formale Sprache L ist genau dann regulär, wenn der Index von $\sim_L$ endlich ist.

Beispiel: Die Sprache $L = \{ab^n : n \geq 0\}$ ist regulär, denn sie hat den Index 3:

- $[\epsilon]_{\sim_L} = \{\epsilon\}$
- $[a]_{\sim_L} = \{a, ab, abb, \dots\} = L$
- $[a]_{\sim_L} = \Sigma^* \setminus (L \cup \{\epsilon\})$

Beispiel: Gegenargument für Regularität: Die Sprache $L = \{a^k b^l c^l : k, l \in \mathbb{N}\} \cup \{b, c\}^*$ ist nicht regulär:

- Betrachte $[u_i]_{\sim_L} = [ab^i c]_{\sim_L}$ für $i \in \mathbb{N}$.
- Mit $w_i = c^{i-1}$ gilt: $u_i w_i = ab^i c^i \in L$ aber es gilt: $u_j w_i = ab^j c^i \notin L$ für $i \neq j$
- Damit sind alle diese Äquivalenzklassen disjunkt und der Index ist unendlich.
- Mit dem Satz von Myhill-Nerode ist die Sprache L nicht regulär.

1.2.5 Notwendige Eigenschaften (Pumping-Lemma)

Um zu zeigen, dass eine Sprache nicht regulär ist, kann das **Pumping-Lemma für reguläre Sprachen** verwendet werden. Auch kontextfreie haben eine ähnliche Variante (in

Abschnitt ??).

Satz: Pumping-Lemma



Jede reguläre Sprache L hat die folgende Pumping-Eigenschaft:

Es gibt eine Zahl $n \in \mathbb{N}_0$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, so dass gilt:

- $|uv| \leq n$
- $|v| \geq 1$
- $\forall i \geq 0: uv^i w \in L$

Das bedeutet nach der klassischen Logik: Erfüllt eine Sprache die Pumping-Eigenschaft nicht, dann ist sie nicht regulär.

Ein Beispiel zum Pumping-Lemma: Zeige, dass $L = \{a^i b^i : i \in \mathbb{N}\}$ nicht regulär ist:

- (i) Der Gegner wählt ein $n \in \mathbb{N}_0$ (also n beliebig)
- (ii) Wir wählen das Wort $z = a^n b^n \in L$ und offensichtlich gilt $|z| \geq n$.
- (iii) Der Gegner wählt eine Zerlegung von $z = uvw$ mit $|uv| \leq n$, $|v| \geq 1$.
- (iv) Dann ist oBdA $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s > 0$ und $w = a^t b^n$ mit $r + s + t = n$.
Dann können wir $i = 2$ wählen und es gilt:

$$uv^i w = a^r a^{2s} a^t b^n = a^{n+s} b^n \notin L$$

1.2.6 Zusammenfassung

- Das Pumping-Lemma kann verwendet werden um die nicht-Regularität zu zeigen.
- Reguläre Ausdrücke erzeugen reguläre Sprachen.
- DFAs und NFAs ohne/mit ϵ -Übergängen erkennen genau die regulären Sprachen.
- Reguläre Sprachen sind unter Schnitt, Vereinigung, Komplement, Kleenschem Abschluss und Produkt abgeschlossen.
- Der Myhill-Nerode Index ist endlich $\iff$ reguläre Sprache.

1.3 Kontextfreie Sprachen

Im Allgemeinen gilt: Kontextfreie Sprachen sind abgeschlossen unter Vereinigung, Produkt und Kleenschem Abschluss. Jedoch unter Schnitt und Komplementbildung **nicht**.

1.3.1 Chomsky-Normalform

Definition: Chomsky-Normalform



Eine kontextfreie Grammatik ist in Chomsky-Normalform, wenn für alle Produktionen P erfüllt ist:

$$P: A \rightarrow w \in \Sigma \text{ oder } A \rightarrow BC \ (B, C \in V)$$

Für die Sprache einer kontextfreien Grammatik kann eine Grammatik in Chomsky-Normalform gefunden werden.

1.3.2 Kellerautomaten

Kellerautomaten haben auch einen Zustandsautomat. Aber zusätzlich wird bei jedem Lesen eines Buchstaben auch der oberste Kellerinhalt gelesen und entsprechend der Produktion verändert und es wird in einen neuen Zustand gewechselt.

Satz: Kellerautomaten für kontextfreie Sprachen



Die kontextfreien Sprachen werden genau von den Kellerautomaten (mit/ohne Endzuständen) erkannt.

Ein Kellerautomat ist deterministisch, wenn für alle Übergänge $\delta(Z, \Sigma, \Gamma)$ gilt:

$$|\delta(z, a, A)| + |\delta(z, \epsilon, A)| \leq 1$$

Das heißt, für jedes Wort hat der DPDA einen eindeutigen Lauf.

1.3.3 Entscheidbarkeit (CYK-Algorithmus)

Wortproblem und Cocke-Younger-Kasami-Algorithmus

Dieser Algorithmus erlaubt es, das Wortproblem für ein Wort aus einer kontextfreien Sprache in Chomsky-Normalform in Polynomialzeit zu entscheiden.

Dazu wird eine linke obere Dreiecksmatrix gezeichnet, mit $|w| = i$ Spalten Zeilen. Im Eintrag $V(i, j)$ wird geprüft, ob es eine Ableitung des Teilworts der Länge j ab dem i -ten Buchstaben gibt. Prüfe dazu die Produktionsmöglichkeiten oberhalb des Eintrags. Gehe jede Zeile darüber durch und finde den dazu passenden Eintrag im Rest der Matrix. Also um $V(3, 4)$ zu bestimmen finde eine Produktion, die das Teilwort der Länge 4 ab Stelle 3 produziert. Dafür kann man durchgehen: $V(3, 1)V(4, 3)$, $V(3, 2)V(5, 2)$ und $V(3, 3)V(6, 1)$. Gibt es eine passende Produktion, trage sie ein. Bei mehreren, trage alle ein.

Paul, hier bitte den Algorithmus bildlich einfügen

1.3.4 Charakterisierungen

Eine Sprache, die auf einem unären Alphabet ($|\Sigma| = 1$) definiert ist, ist genau dann kontextfrei, wenn sie regulär ist.

Daraus folgt, dass die folgenden Sprachen nicht kontextfrei sind (da sie nicht regulär sind):

- $L_1 := \{a^p : p \text{ Primzahl}\}$
- $L_2 := \{a^p : p \text{ keine Primzahl}\}$
- $L_3 := \{a^n : n \text{ Quadratzahl}\}$
- $L_4 := \{a^{2^n} : n \in \mathbb{N}\}$

1.3.5 Notwendige Eigenschaften (Pumping Lemma)

Pumping-Lemma

Satz: Pumping-Lemma

Jede kontextfreie Sprache L hat die folgende Pumping-Eigenschaft:

Es gibt eine Zahl $n \in \mathbb{N}_0$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvwxy$ geschrieben werden kann, so dass gilt:

- $|vwx| \leq n$
- $|vx| \geq 1$
- $\forall i \geq 0: uv^iwx^iy \in L$

Das bedeutet nach der klassischen Logik: Erfüllt eine Sprache die Pumping-Eigenschaft nicht, dann ist sie nicht kontextfrei.

Die Umkehrung gilt nicht. Nicht jede Sprache, die das Pumping-Lemma erfüllt, ist auch kontextfrei.

Der Beweis ist eine gute Eselsbrücke für das Lemma

1.3.6 Zusammenfassung

1.4 Kontextsensitive Sprachen

Satz: Satz von Kuroda

Die kontextsensitiven Sprachen werden von den nichtdeterministischen linear beschränkten Turingmaschinen erkannt.

Allgemein werden alle Typ-0 Sprachen genau von den nichtdeterministischen Turingmaschinen erkannt. Dabei spielt die lineare Beschränktheit erst für die Laufzeiten eine Rolle.

Der Satz von Immermann besagt, dass die kontextsensitiven Sprachen abgeschlossen unter Komplementbildung sind.

1.5 Allgemeine Sprachen

1.5.1 Turing-Maschinen

1.5.2 Entscheidbarkeit

Definition: Definition

Eine Sprache $L \subseteq \Sigma^*$ heißt **entscheidbar/rekursiv**, wenn die charakteristische Funktion von L :

$$\chi_L: \Sigma^* \rightarrow \{0, 1\}, \omega \mapsto \begin{cases} 1 & \omega \in L \\ 0 & \omega \notin L \end{cases}$$

berechenbar ist.

Dazu eng verwandt ist die semi-Entscheidbarkeit: Eine Sprache heißt **semi-entscheidbar/rekursiv aufzählbar**, wenn die charakteristische Funktion für $\omega \notin L$ undefiniert ist. Die semi-entscheidbaren Sprachen sind die rekursiv aufzählbaren Sprachen.

Satz: Kriterium für Entscheidbarkeit

Eine Sprache L ist genau dann entscheidbar, wenn L und $\bar{L}$ semi-entscheidbar sind.
Eine Sprache L ist genau dann entscheidbar, wenn $\bar{L}$ entscheidbar ist.

Staatsexamensaufgabe (F25T1A3 (nicht-vertieft))

Für eine Sprache L (übers Alphabet Σ) wird mit $\bar{L}$ ihr Komplement, d.h. $\bar{L} = \Sigma^* \setminus L$ bezeichnet. Sind die folgenden Aussagen korrekt? Begründen Sie jeweils Ihre Antwort.

Lösungsvorschlag: Lösung für die Aufgabe?

1.5.3 Halteprobleme**Satz: Spezielles Halteproblem**

Das spezielle Halteproblem ist die Sprache

$$K := \{\omega \in \{0, 1\}^* \mid M_\omega \text{ hält für die Eingabe } \omega\}$$

und ist **nicht** entscheidbar.

Satz: Allgemeines Halteproblem

Das allgemeine Halteproblem ist die Sprache

$$H := \{\omega \# x \mid M_\omega \text{ hält für die Eingabe } x\}$$

Es ist **nicht** entscheidbar.

1.5.4 Reduktionsbeweise

Dass eine Sprache entscheidbar oder unentscheidbar ist, ist häufig nicht einfach von Grund auf zu Beweisen. Dafür gibt es die Reduktion:

Definition: Reduktion

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprache. Dann sagen wir L_1 ist auf L_2 reduzierbar, geschrieben $L_1 \leq L_2$, falls es eine totale und berechenbare Funktion $f: \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $\omega \in \Sigma_1^*$

$$w \in L_1 \iff f(w) \in L_2$$

Satz: Reduktionsbeweis

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.
Die Kontraposition: Wenn $L_1 \leq L_2$ und L_1 ist unentscheidbar, dann ist auch L_2 unentscheidbar.

Das Halteproblem auf leerem Band H_0 ist unentscheidbar. Das folgt aus Reduktion auf das allgemeine Halteproblem.

1.5.5 NP-Vollständigkeit

Wir haben bereits gelernt, dass verschiedene Algorithmen unterschiedlich lange benötigen und dass wir die worst-case Dauer eines Algorithmus mit der O -Notation in Relation zur Eingabegröße angeben können. Haben wir beispielsweise eine unsortierte Liste von Größe n gegeben, so ist die Komplexität, also die Laufzeit des BubbleSort-Algorithmus $O(n^2)$. Der Quicksort-Algorithmus hat hingegen eine Laufzeit von $O(n \log n)$ was für große Listen einen riesigen Unterschied macht.

Es gibt aber Klassen von Problemen in der Informatik, bei denen wir nur von einer Komplexität $O(n^k)$ träumen können. Diese Probleme sind so schwer, dass sie nur in Exponentialzeit $O(a^n)$ lösbar sind. Ein klassisches Beispiel ist das Travelling Salesman Problem: Ein Händler will alle Großstädte in Deutschland besuchen. Was ist der kürzeste Pfad, bei dem er jede Stadt genau einmal besucht und am Ende zum Ursprungspunkt zurückkehrt? Was die Lösung dieses Problems schwierig macht, ist dass selbst die besten Lösungsalgorithmen kaum besser als Brute Force sind.

Probleme, die diese Eigenschaft haben, werden als **NP-vollständig** bezeichnet. Wie können wir also herausfinden, ob ein Problem in Polynomialzeit lösbar ist oder nicht? Es stellt sich heraus, dass es gar nicht leicht ist, direkt zu zeigen, dass ein Problem NP-schwer ist. Wir können aber zeigen, dass ein Problem genauso schwer ist wie ein Problem von dem wir wissen, dass es **NP-vollständig** ist.

Wenn man etwas Mathematik gemacht hat, weiß man, dass man eine Gleichheit zeigt, indem man die zwei Ungleichungen $A \leq B$ und $A \geq B$ beweist. Sprich, wir zeigen, dass A leichter ist als B (**NP**) und gleichzeitig, dass A schwerer ist als B (**NP-schwer**).

Um die Diskussion auf ein formelles Fundament zu stellen, tun wir so als ob jedes Problem, dass sich uns stellen könnte auf die Frage heruntergebrochen werden kann, ob ein Wort w in einer Sprache L liegt.

Definition: NP

Eine Sprache L heißt NP, falls eine der folgenden Bedingungen erfüllt ist:

- Es gibt eine nichtdeterministische Turingmaschine, die in polynomieller Zeit erkennt, ob ein Wort w in L enthalten ist.
- Angenommen $w \in$

Definition: NP-Schwere

Eine Sprache L heißt **NP-schwer**, falls alle anderen NP Probleme auf L reduzierbar sind.

Definition: NP-Vollständigkeit

Eine Sprache L ist **NP-vollständig**, wenn sie **NP** und **NP-schwer** ist.

Es ist wichtig zu verstehen, dass NP-vollständige Sprachen leichter als alle NP-schweren Sprachen aber schwerer als alle NP-Sprachen sind.

Ein explizites Problem angeben, wo Brute force zu einer exponentiellen Laufzeit führt

Ein Problem angeben, das in Polynomialzeit lösbar ist aber eine einfache exponentielle Lösung erlaubt

Explizite Beispiele geben

Diese Entscheidung legitimieren.

Mathcal-O

Es gibt die Cha-

Aus der Definition geht bereits hervor, dass wir für den Beweis von NP-Schwere eine Reduktion benötigen. Diese verläuft folgendermaßen: Angenommen, das Problem A ist leicht zu lösen, dann können wir zeigen, dass ein anderes Problem B in NP leicht zu lösen. Damit muss B leichter sein als A .

Darauf eingehen, dass die Reduktion anders als die Reduktion für Unentscheidbarkeit ist.

Staatsexamensaufgabe (F25T1T2A2)

Beantworten Sie die folgenden Fragen und begründen Sie Ihre Antwort. Sei P die Klasse der Entscheidungsprobleme, die deterministisch in Polynomialzeit lösbar sind.

- (a) Skizzieren Sie, wie sich TSPW in Polynomialzeit lösen lässt, wenn TSP in P liegt.

TSP	
Gegeben:	Gewichteter Graph G , Zahl $k \in \mathbb{N}$
Frage:	Hat G einen Kreis mit Gewicht $\leq k$, der jeden Knoten genau einmal besucht?
TSPW	
Gegeben:	Gewichteter Graph G
Frage:	Was ist das minimale Gewicht eines Kreises von G , der alle Knoten genau einmal besucht?

- (b) Im Allgemeinen ist das k -COL-Problem für ein $k \in \mathbb{N}$ wie folgt definiert: Sie dürfen annehmen, dass 5-COL NP-vollständig ist. Beweisen Sie, dass 3-COL NP-schwer ist. Geben Sie dafür eine Reduktion von 3-COL auf 5-COL an.

k -COL	
Gegeben:	Ein Graph $G = (V, E)$
Frage:	Kann jeder Knoten V so gefärbt werden, dass nur k Farben verwendet werden und Knoten $v_1, v_2 \in V$ mit $(v_1, v_2) \in E$ immer unterschiedlich gefärbt sind?

- (c) Ist das folgende Entscheidungsproblem entscheidbar?

TM-Halt	
Gegeben:	Turingmaschine M in üblicher Tupeldarstellung, natürliche Zahl k
Frage:	Hält M auf allen Eingaben x nach spätestens $2k + 1$ Schritten?

Lösungsvorschlag:

- (a) Im folgenden gehen wir davon aus, dass der Graph G nur Kantengewichte in $\mathbb{N}$ hat. Wir wissen, dass TSP in Polynomialzeit lösbar ist. Für einen gegebenen Graphen G und eine Zahl $k \in \mathbb{N}$ lässt sich also in Polynomialzeit feststellen, ob ein Kreis mit Gewicht $\leq k$ der jeden Knoten genau einmal besucht. Will man das minimale Gewicht eines Kreises finden, der alle Knoten besucht, dann kann man mithilfe TSP berechnen, ob es einen Kreis mit Gewicht 0 gibt. Wenn nicht, prüft man ob es einen Kreis mit Gewicht $\leq 1, \leq 2$, usw. gibt. Gibt es für k einen solchen Kreis aber nicht für $k' < k$, dann haben wir eine Antwort auf das TSPW gefunden. Wir beobachten anschließend, dass der Algorithmus nicht terminiert, wenn der Graph keinen solchen Kreis besitzt. Um auszuschließen, dass der Algorithmus kein Ende hat, wollen wir nur bis zu einer bestimmten Zahl $K \in \mathbb{N}$ testen. Wir wählen K als die Summe aller Kantengewichte. Das ist ausreichend, da ein Kreis, der jeden Knoten

genau einmal besucht jede Kante maximal einmal traversiert (andernfalls würde er einen Knoten zweimal besuchen). Das Gewicht des Kreises ist folglich kleiner als die Summe aller Kantengewichte K . Besitzt der Graph also keinen solchen Kreis für $\leq K$, dann hat er gar keinen solchen Kreis.

Insgesamt führt der Algorithmus maximal K mal TSP aus und K wächst linear mit der Zahl der Knoten.

- (b) Um zu zeigen, dass 5-COL NP-schwer ist müssen wir zeigen, dass es polynomiell von 3-COL abhängt. Das geht wie folgt: Angenommen, 5-COL wäre in Polynomialzeit lösbar. Ist dann 3 ebenfalls in Polynomialzeit lösbar, dann haben wir die Abhängigkeit gezeigt.

Angenommen, 5-COL wäre in Polynomialzeit lösbar. Wir wollen herausfinden ob die Knoten eines Graphs G mit 3 Farben gefärbt werden können, sodass keine benachbarten Knoten die gleiche Farbe haben. Die typische Strategie ist es, den Graphen angemessen zu modifizieren. [Beobachte erstmal, dass ein Knoten eine in einem vollständigen Graphen (jeder Knoten ist mit jedem anderen Knoten verbunden) jeder Knoten eine andere Farbe haben muss.] Wir fügen zum Graphen G zwei neue Knoten v_1, v_2 hinzu. Diese Knoten verbinden wir mit jedem anderen Knoten des Graphen und wir verbinden die beiden Knoten miteinander. Die Knoten müssen damit eine andere Farbe als alle anderen Knoten haben. Besitzt der Graph $G \cup \{v_1, v_2\}$ nun also eine 5-Färbung, dann erhalten wir durch das Entfernen der beiden Knoten eine 3-Färbung auf G . Besitzt G eine 3-Färbung, dann erhalten wir durch das Hinzufügen der Knoten eine 5-Färbung. Die beiden Probleme sind also äquivalent, und die Lösung von 5-COL gibt eine Lösung von 3-COL.

- (c) Der erste Instinkt ist, dass das Entscheidungsproblem nicht entscheidbar ist (es sieht so aus, als könnte man es auf das Halteproblem reduzieren). Auf den zweiten Blick scheint es aber möglich zu sein, zu überprüfen, ob eine Turingmaschine nach $2k + 1$ Schritten hält. Schließlich müssen wir bloß $2k + 1$ Schritte durchführen und dann schauen, ob die Turingmaschine in der Zeit gehalten kann. Allerdings ist es unmöglich dies für **alle** x zu prüfen. Auf den dritten Blick sieht das Problem also wieder unlösbar aus. Da die ersten zwei Ansätze nicht funktioniert haben probiere die Aufgabe mit dem Satz von Rice zu lösen. Charakterisiere die Aufgabenstellung mit der Funktion

$$f_M(x) = \begin{cases} 1 & M \text{ hält nach } n \leq 2k + 1 \text{ Schritten} \\ 0 & M \text{ hält nicht nach } n \leq 2k + 1 \text{ Schritten} \end{cases}$$

Die Funktion ist für jedes x berechenbar, denn man braucht maximal $2k + 1$ Schritte von M um das Ergebnis festzustellen. Der Satz von Rice besagt nun, dass die Sprache der Maschinen, die f_M berechnen unentscheidbar ist.

□ Theoretisch könnte 5-COL auch einfach so polynomiell sein, den schließlich wissen wir NP/P nicht.

Das ist der richtige Weg aber leider nicht die vollständige Lösung.

1.5.6 Satz von Rice

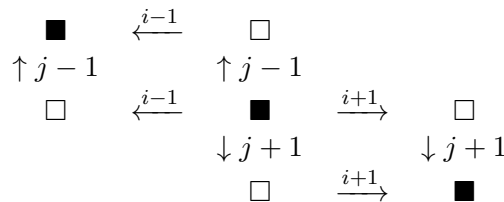
Satz: von Rice



Sei $\mathfrak{R}$ die Menge der Turingberechenbaren Funktionen. Sei $\mathfrak{S}$ eine Teilmenge $\emptyset \subset \mathfrak{S} \subset \mathfrak{R}$. Dann ist

$$C(\mathfrak{S}) = \{\omega \mid \text{Die von } M_\omega \text{ berechnete Funktion liegt in } \mathfrak{S}\}$$

eine unentscheidbare Sprache.



1.6 Berechenbarkeit

Definition: Turingberechenbarkeit

Eine Funktion $f: \Sigma^* \rightarrow \Sigma^*$ ist Turingberechenbar, wenn gilt:

$$f(u) = v \iff z_0 u \vdash^* \square \dots \square z_e v \square \dots \square, \quad z_e \in E$$

Die LOOP-Programme sind immer totale Funktionen. Es gibt Turingberechenbare Funktionen, die aber nicht LOOP-Berechenbar sind (zum Beispiel die überall undefinierte Funktion) oder die Busy Beaver Funktion.

Addition, Multiplikation und andere Rechenarten sind LOOP-Berechenbar. Auch IF-Programme sind LOOP-berechenbar.

LOOP-berechenbare Funktionen sind auch WHILE-berechenbar und WHILE-berechenbare Funktionen sind auch Turing-berechenbar.

$$\text{Turing-berechenbar} \iff \text{WHILE-berechenbar} \iff \text{GOTO-berechenbar}$$

Die Ackermann-Funktion ist ein klinisches Beispiel für eine totale Funktion, die WHILE-berechenbar ist (zum Beispiel, weil sie leicht in einer modernen Programmiersprache zu implementieren ist), aber nicht LOOP-berechenbar ist.

Die primitiv rekursiven Funktionen werden genau von LOOP-Programmen berechnet.

Die μ -rekursiven Funktionen entsprechen genau den WHILE-berechenbaren Funktionen.

Die Busy Beaver Funktion habe ich noch nie im Stex gesehen aber die ist echt lustig und nicht so schwer zu verstehen.

1.7 Zusammenfassung

Bei den Sprachen erhalten wir die folgende Hierarchie:

1. Typ 3/reguläre Sprache
2. Typ 2/kontextfreie Sprache (z.B. $\{a^n b^n \mid n \in \mathbb{N}\}$)

3. Typ 1/kontextsensitive Sprache (z.B. $\{a^n b^n c^n \mid n \in \mathbb{N}\}$)
4. Entscheidbare Sprache/Rekursive Sprache
5. Typ 0/semientscheidbare Sprache/Rekursiv aufzählbare Sprache (z.B. das Halteproblem)
6. Absolutes Chaos (z.B. $\{M_1 \# M_2 \mid M_1, M_2 \text{ haben die gleichen Ausgaben}\}$)

Bei Funktionen ergibt sich folgende Hierarchie:

1. LOOP-berechenbare/Primitiv-rekursive Funktion (z.B. Fibonacci-Funktion)
2. WHILE-berechenbare/ μ -rekursive Funktion (z.B. Ackermann-Funktion)
3. Chaos (z.B. Busy Beaver)

Matthias, bitte hier die Übersicht aus VL8 einfügen!

2 Algorithmen und Datenstrukturen

Der Begriff Algorithmus fällt nicht all zu selten im Kontext jeglicher Situationen mit informatischen Gehalt. Dabei wird ein Algorithmus nicht selten darauf reduziert, welches Ergebnis er letztendlich liefert. Besonders häufig werden dabei Empfehlungs- und Wegfinderalgorithmen im gesellschaftlichen Diskurs thematisiert. Es wird nicht die Frage gestellt, *wie* der Algorithmus etwas produziert, sondern *was* dabei herauskommt. Der Algorithmusbegriff entspricht gesellschaftlich damit häufig einer Black-Box, die etwas produziert, aber niemand weiß, was hinter diesem ominösen Algorithmus steckt.

Die Informatik versteht dabei deutlich mehr unter dem Begriff des Algorithmus. Sie beschreibt Algorithmen als Abfolge von Anweisungen, die zur Lösung eines Problems oder einer ganzen Problemklasse dient. Ein Algorithmus beschreibt als den Weg vom Problem hin zur Lösung. Im Zuge dessen ist aber die Vorstellung des Outputs nicht ganz unsinnvoll. Die Vorstellung eines Alorithmus als Funktion, die etwas berechnet ist dabei sogar die richtige. Jedoch interessiert sich die Fchinformatik für alles (und nicht nur den Output): Die Eingabe, die Ausgabe, den Weg dorthin, die Effizienz des Wegs und auch die unterliegende Datenstruktur, die den Problemraum beschreibt.

Dieses Kapitel klärt zuerst grundlegende Begriffe rund um Algorithmen. Danach werden Datenstrukturen vorgestellt, die hier eine Doppelrolle spielen: Einerseits werden Datenstrukturen entwickelt und vorgestellt, die einen Alporithmus erst effizient machen. Andererseits wird auch die problemraumbestimmende Rolle der Datenstruktur für einen Algorithmus beleuchtet.

- Groß-O Notation
- Listen: Sortiervverfahren und deren Laufzeiten/Eigenschaften
- Bäume: Operationen auf Bäumen, Suchverfahren und Eigenschaften incl Laufzeiten
- Graphen: Dijkstra, Jarnik-Prim, Kürzeste Wege, minimale Spannbäume,
- Hashing, Streuspeicherung

2.1 Grundlagen zu Algorithmen

Wie in der Einleitung geschildert, soll ein Algorithmus ein Problem lösen bzw. etwas berechnen. Im Berechnungsfall ist das zugehörige Problem die Fragestellung nach dem Ergebnis des Algorithmus. Aber was ist eigentlich wirklich ein Algorithmus?

Der Algorithmusbegriff lässt sich dabei wie folgt definieren:

Definition: Algorithmus



Ein Algorithmus ist

Muwss man in diesem Kapitel angesichts unserer Zielgruppe erst das Programmieren beibringen oder kann man das voraussetzen?

2.1.1 Eigenschaften von Algorithmen

Aus dieser Definition lassen sich bestimmte Eigenschaften ableiten, die bei Algorithmen einzuhalten sind und bei einem existierenden Algorithmus als erfüllt anzunehmen sind. Nach der Definition werden alle fünf Eigenschaften ausführlich erklärt.

Definition: Eigenschaften von Algorithmen

Jeder Algorithmus ist:

- allgemein
- determiniert
- deterministisch
- terminierend
- partiell korrekt

Allgemeinheit von Algorithmen

Die Allgemeinheit eines Algorithmus beschreibt die Eigenschaft, dass ein Algorithmus die Lösung einer ganzen Klasse von Problemen ist. Damit ist gemeint, dass unabhängig von der gewählten Programmiersprache oder den explizit vorliegenden Daten, der Algorithmus ein bestimmtes Problem lösen kann.

Ein klassisches Beispiel dafür sind die in Kapitel vorgestellten Sortierverfahren. Ein (Sortier-)Algorithmus wie *BubbleSort* sortiert eine gegebene Liste an Daten. Ob das nun ganze Zahlen, Listen mit Büchern einer Bibliothek oder Namen von Schulkindern einer Klasse sind, ist prinzipiell egal. Wichtig ist, dass auf der Datenstruktur eine Ordnungsrelation definiert ist. Ebenfalls ist es irrelevant in welcher Programmiersprache der Algorithmus verfasst ist. Wichtig ist die *Idee* hinter dem Algorithmus. Also nicht sehr präzise gesprochen: Was soll ein Sortieralgorithmus tun? Irgendwas sortieren, egal was!

Hier Kapitelref

Determiniertheit von Algorithmen

Dass ein Algorithmus determiniert (in etwa interpretierbar als: im Vorhinein festgelegt) ist bedeutet, dass er für die gleiche Eingabe die gleiche Ausgabe liefert. Diese Eigenschaft schließt zu aller erst aus, dass der Algorithmus (pseudo-)zufällige Elemente enthält.

Diese Eigenschaft ist wichtig, weil damit die Reproduzierbarkeit von Ergebnissen gewährleistet ist. In vielen Bereichen wäre ein Algorithmus, der bei gleicher Eingabe unterschiedliche Ergebnisse liefert absolut unbrauchbar. Dazu stelle man sich folgenden (einfachen) Algorithmus vor: Man kann als Eingabe das Datum tätigen und erhält den richtigen Wochentag als Rückgabe. Wenn Sie also den Wochentag für Heiligabend dieses Jahr ermitteln wollen und ständig ein anderes Ergebnis erhalten wäre das absurd. Sie erwarten also, dass beispielsweise für den 24.12.2025 immer das Ergebnis "Mittwoch" geliefert wird.

Determinismus von Algorithmen

Der Determinismus ist eng verwandt mit der Determiniertheit. Jedoch beschreibt der Determinismus, dass ein Algorithmus bei gleicher Eingabe immer den gleichen Lauf hat. Der Begriff des Laufs wird in der Literatur zur Spezifikation und Verifikation von Algorithmen näher diskutiert. Wichtig zu verstehen ist nur: Wenn die gleiche Eingabe getätigt wird,

dann durchläuft der Algorithmus immer exakt die gleichen Schritte auf die gleiche Art und Weise.

Terminierung von Algorithmen

Die Eigenschaft, dass ein Algorithmus terminierend sein muss beschreibt, dass der Algorithmus nach einer endlichen Zahl von Schritten halten soll (und eine Ausgabe geben soll). Ist diese Eigenschaft verletzt, so bedeutet das, dass der Algorithmus nie aufhört und somit auch keine Lösung für das behandelte Problem/Rechenaufgabe liefert.

Partielle Korrektheit von Algorithmen

Die partielle Korrektheit eines Algorithmus besagt, dass die versprochene Ausgabe (zum Beispiel eine fertig sortierte Liste) beim Halten des Algorithmus auch tatsächlich vorliegt. Diese Eigenschaft ist bedeutend, da vor allem hochsensible Systeme keinen Raum für Fehler lassen und daher der Algorithmus auch keine falschen Ausgaben produzieren darf. Mit bestimmten logischen Verfahren ist es möglich die partielle Korrektheit zu beweisen. Das ist aber ebenfalls ein Teilgebiet der formalen Spezifikation und Verifikation.

2.1.2 Laufzeiten und Komplexitätsklassen

Da nun bekannt ist, was ein Algorithmus ist, kann man sich im Folgenden nun die Frage stellen: Wenn ich also einen Algorithmus entwickelt habe, der ein bestimmtes Problem löst ... *wie lange* benötigt der Algorithmus, bis die Lösung berechnet ist?

Um diese Frage zu beantworten, sollte man sich die Fragen stellen, wovon es eigentlich abhängt, wie lange ein Algorithmus benötigt um zu halten. Dafür gibt es mehrere Einflüsse:

- Wie viele Datenpunkte umfasst die Datenstruktur, auf der der Algorithmus operiert?
⇒ Die Anzahl der Datenpunkte wird als n bezeichnet.
- Welche Schleifen durchläuft der Algorithmus pro Datenpunkt?
- Gibt es einen Worst-Case, bei dem maximal viele Schritte benötigt werden (*in Abhängigkeit von n*)?

Beispiel: Die Maximumsfunktion

Um die Laufzeiten zu motivieren und noch keine allzu komplizierte Notation zu verwenden betrachte man folgenden Algorithmus: Der Algorithmus erhält eine Liste mit ganzen Zahlen der Länge n als Eingabe. Die Aufgabe ist es, als Ausgabe das größte Element zurückzugeben. Dann kann der Algorithmus wie folgt (als Pythoncode) ablaufen:

```
1 def maximum(input:list[int]) -> int:
2     # Das erste Element wird als Maximum gesetzt
3     max = input[0]
4     # In jedem Durchlauf wird geprüft, ob das betrachtete Element größer ist,
5       als das aktuelle Maximum.
6     # Wenn das der Fall ist, wird das Maximum ersetzt.
7     for i in input:
8         if i > max: max = i
9     return max
```

Um die Fragen nach der Laufzeit zu beantworten, kann man den Code jetzt genau unter die Lupe nehmen: Die Eingabe ist wie immer die Länge n . Zuerst wird das allererste Element als Maximum gesetzt. Dabei ist es egal, wie lang die Liste ist, dieser Vorgang dauert immer gleich lang. Man formuliert: *"Dieser Schritt läuft in konstanter Zeit ab."* Die Referenz bildet bei all diesen Aussagen immer die Länge der Eingabe.

Danach wird die Liste *einmal* durchgegangen. Das bedeutet diese Schleife benötigt so viele Schritte, wie die Liste lang ist. Demnach n Schritte. Aber wichtig ist noch, was in der Schleife passiert. In jedem Schritt der Schleife wird eine Vergleichsoperation durchgeführt und eventuell die Variable `max` neu belegt. Diese Vorgänge sind aber alle wieder unabhängig von der Länge der Liste, die Schritte laufen also in konstanter Zeit ab.

Daher kann man sagen: *"Die Schleife läuft in linearer Zeit ab."* Da die Schleife den größten Einfluss auf die Laufzeit des Beispielsalgorithmus hat, kann man den Algorithmus den Algorithmen mit *linearer Laufzeit* zuordnen. Man schreibt:

$$\text{maximum} \in \mathcal{O}(n)$$

Zurecht könnte man sich fragen, ob da nicht etwas übersehen wurde: In den Überlegungen zur Laufzeit wurde formuliert, dass der Worst-Case betrachtet werden soll. Für die Maximumsfunktion wäre der schlimmste Fall, wenn die Liste von klein nach groß sortiert ist und in jedem Schleifendurchlauf das alte Maximum ersetzt werden muss. Dann benötigt der Algorithmus einen Schritt für das erstmalige Setzen des Maximums. Dann $2 \cdot n$ Schritte für den Schleifendurchlauf (1. Schritt: Vergleichen, 2. Schritt: Neues Maximum setzen). Und zum Schluss noch einen Schritt für die Ausgabe. Insgesamt kommt man so auf die Schrittzahl von:

$$1 + 2n + 1 = 2(n + 1)$$

Sollte es dann nicht eigentlich heißen: $\text{maximum} \in \mathcal{O}(2(n + 1))$?

Einen Algorithmus auf diese Weise in seiner Laufzeit zu beschreiben mag auf den ersten Blick geeigneter sein als in der Form davor. Jedoch würde diese Schreibweise keinen Platz für viele Fallunterscheidungen und Schleifen in Schleifen bieten. Einen etwas längeren Algorithmus zu beschreiben wäre damit im Grunde genommen unmöglich. Daher haben sich die Informatiker darauf geeinigt, sogenannte *Laufzeitklassen* einzuführen.

Definition der Laufzeitklassen

Die Idee hinter den Klassen ist es, Algorithmen bezüglich ihrer Laufzeit im Unendlichen zu sortieren (unendlich lange Eingaben). Dafür wurde folgende Definition eingeführt, die auch *Landau-Notation* heißt:

$$\mathcal{O}(g) = \{f : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |f(x)| \leq c \cdot |g(x)|\}$$

Leichter zu verstehen ist die Definition mit dieser äquivalenten Schreibweise:

$$\mathcal{O}(f) = \left\{ g : \mathbb{R} \rightarrow \mathbb{R} : \limsup_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| < \infty \right\}$$

Für das Beispiel der Maximumsfunktion gilt also: Der Algorithmus läuft in Linearzeit ab ($\mathcal{O}(n)$), weil er im Unendlichen nur um einen Faktor kleiner als Unendlich schneller wächst als ein Algorithmus mit genau n Schritten bei einer Eingabe mit Länge n .

Mathematisch berechnen und damit zeigen kann man das mit der Überlegung von oben wie folgt:

$$\limsup_{n \rightarrow \infty} \left| \frac{2(n+1)}{n} \right| = 2 \cdot \limsup_{n \rightarrow \infty} \left| \frac{n+1}{n} \right| = 2 \cdot \limsup_{n \rightarrow \infty} |1| = 2 < \infty$$

In Worten kann man die Einsortierung in Laufzeitklassen auch beschreiben: Eine Funktion/Algorithmus ist in einer Laufzeitklasse $\mathcal{O}(g)$ (geschrieben $f \in \mathcal{O}(g)$), wenn das Wachstumsverhalten von f langsamer oder genauso schnell ist wie das von g .

Eine strengere Form des Wachstums wird mit folgender Definition beschrieben (man beachte die Ersetzung des Existenzquantors vor c durch einen Allquantor):

$$o(f) = \{g: \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |g(x)| \leq c \cdot |f(x)|\}$$

Man schreibt $f \in o(g)$, wenn das Laufzeitverhalten von f echt langsamer ist als das von g . Ebenfalls leichter verständlich ist die Definition mithilfe des Limes:

$$o(f) = \left\{ g: \mathbb{R} \rightarrow \mathbb{R} : \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = 0 \right\}$$

In der Informatik werden im Großen und Ganzen diese vier Laufzeitklassen unterscheiden:

Definition: Laufzeitklassen

$$\mathcal{O}(g) = \{f: \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |f(x)| \leq c \cdot |g(x)|\}$$

$$o(f) = \{g: \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |g(x)| \leq c \cdot |f(x)|\}$$

$$\Omega(g) = \{f: \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0 : \exists x_0 > 0 : \forall x > x_0 : c \cdot |g(x)| \leq |f(x)|\}$$

$$\Theta(g) = \{f: \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0 : \exists C > 0 : \exists x_0 > 0 : \forall x > x_0 : c \cdot |g(x)| \leq |f(x)| \leq C \cdot |g(x)|\}$$

Rechenregeln für die Landau-Notation

Um längere und zusammengesetzte Algorithmen leichter einer Klasse zuzuordnen, gibt es bestimmte Rechenregeln für die Landau-Notation. Im Folgenden sind f und g Funktionen und $a \in \mathbb{R}$ ein beliebiger Faktor:

- $f + g \in \mathcal{O}(\max(f, g))$
- $f \cdot g \in \mathcal{O}(f \cdot g)$
- $g \in \mathcal{O}(f) \iff a \cdot g \in \mathcal{O}(f)$
- Logarithmen unterschiedlicher Basen fallen in die gleiche Klasse

Rekursionsformeln

Die Laufzeit rekursiver „Divide-and-Conquer-Funktionen“ kann durch folgende Eigenschaft berechnet werden:

Wie schreiben wir Examensaufgaben?

Rekursionsformel: Divide-and-Conquer (Mastertheorem)

Für eine Rekursionsgleichung der Form

$$T(n) = \begin{cases} c & n \leq 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases}$$

mit $c > 0, a > 0, b > 1, f(n) \in \mathcal{O}(n^d), d > 0$ gilt:

$$T(n) \in \begin{cases} \mathcal{O}(n^d) & d > \log_b(a) \\ \mathcal{O}(n^d \log(n)) & d = \log_b(a) \\ \mathcal{O}(n^{\log_b(a)}) & d < \log_b(a) \end{cases}$$

Bei der Subtract-and-Conquer Regel gilt die folgende Eigenschaft zur Laufzeit:

Rekursionsformel: Subtract-and-Conquer

Für eine Rekursionsgleichung der Form

$$T(n) = \begin{cases} c & n \leq 1 \\ a \cdot T(n - b) + f(n) & n > 1 \end{cases}$$

mit $c > 0, a > 0, b > 1, f(n) \in \mathcal{O}(n^d), d \geq 0$ gilt:

$$T(n) \in \begin{cases} \mathcal{O}(n^d) & a < 1 \\ \mathcal{O}(n^{d+1}) & a = 1 \\ \mathcal{O}(n^d a^{\frac{n}{b}}) & a > 1 \end{cases}$$

In vielen Fällen kann auch eine vollständige Induktion genutzt werden, um Laufzeiten von rekursiven Formeln zu berechnen.

2.2 Listen und Operationen auf Listen

Arten von Listen

Für Listen gibt es klassischerweise einige Beispiele. Das sind:

- Einfache Arrays
- Einfach verkettete Listen (Jeder Knoten kennt nur seinen Nachfolger)
- Doppelt verkettete Listen (Jeder Knoten kennt Vorgänger und Nachfolger)
- Einfach verankerte Listen (Start bekannt, Rest einfach verkettet)
- Doppelt verankerte Listen (Start und Ende bekannt, Rest einfach verkettet)
- Stacks (nach dem LIFO-Prinzip): Es kann immer nur auf das oberste Objekt zugegriffen werden, der Rest ist wieder einfach verkettet
- Queue (nach dem FIFO-Prinzip): Es kann nur am Ende eingefügt werden und auf den Anfang zugegriffen werden, der Rest ist einfach verkettet

2.2.1 Sortierverfahren

Die gängigen Sortierverfahren können den folgenden Komplexitätsklassen zugeordnet werden:

- $\mathcal{O}(n^2)$: BubbleSort, SelectionSort, InsertionSort
- $\mathcal{O}(n \log(n))$: MergeSort, QuickSort, HeapSort

In der Vorlesung wurde gezeigt:

Die Anzahl der benötigten Vergleiche in jedem vergleichsbasierten Sortierverfahren wächst im Worst-Case mindestens wie $n \log(n)$.

Kommentare
für die
Codes

BubbleSort

Die Idee ist, dass die großen Einträge wie Luftblasen immer weiter aufsteigen. Dabei wird die Liste immer von vorne durchgegangen aber immer einen Schritt kürzer, da im ersten Schritt ja das größte Element bis ganz nach oben gestiegen ist. Somit bleibt die rechte Seite des Arrays sortiert. Abbildung ?? zeigt eine Implementierung in Pseudo-Code.

```
1 int[] bubbleSort(int[] liste)
2   for start in {0, ..., länge(liste)-1}
3     for n in {start, ..., länge(liste)-2}
4       if liste[n] > liste[n+1]
5         tausche(liste[n], liste[n+1])
6   return liste
```

Abbildung 2.1: BubbleSort in Pseudo-Code

SelectionSort

Die Idee ist, dass der erste Eintrag als Schlüssel gespeichert wird und dann das Minimum der rechten Seite gesucht wird. Dann wird der Schlüssel mit dem Minimum vertauscht oder gleich gelassen, wenn der Schlüssel das Minimum war. Danach wird das nächste Element als Schlüssel gewählt und wieder das neue Minimum gesucht. Somit bleibt die linke Teilliste immer sortiert. Abbildung ?? zeigt eine Implementierung in Pseudo-Code.

```
1 int[] selectionSort(int[] liste)
2   int kleinstenWertIndex
3   for start in {0, ..., länge(liste)-1}
4     kleinstenWertIndex = start
5     for n in {start, ..., länge(liste)-1}
6       if liste[n] < liste[kleinstenWertIndex]
7         kleinstenWertIndex = n
8     tausche(liste[start], liste[kleinstenWertIndex])
9   return liste
```

Abbildung 2.2: SelectionSort in Pseudo-Code

InsertionSort

Die Idee ist, dass die Liste von einem Startelement nach rechts durchgegangen wird, bis ein Element gefunden wird, das die Ordnung verletzt. Dieses wird so lange nach vorne verglichen, bis der richtige Platz gefunden ist. Dann wird der Laufindex um eins erhöht und die Schleife beginnt erneut. Also wird die linke Teilliste immer sortiert gehalten.

Die Laufzeit ist im Average-Case und Worst-Case in $\mathcal{O}(n^2)$ und im Best-Case $\mathcal{O}(n)$ bei vorsortierter Liste. Abbildung ?? zeigt eine Implementierung in Pseudo-Code.

```

1  int[] insertionSort(int[] liste)
2      for ende in {1, ..., länge(liste)}
3          einzusortierendesElement = liste[ende]
4          n = ende
5          while n>0 and liste[n] > einzusortierendesElement
6              liste[n] = liste[n-1]
7              n = n-1
8          liste[n] = endeElement
9  return liste

```

Abbildung 2.3: InsertionSort in Pseudo-Code

MergeSort

Hier wird die Strategie Divide-and-Conquer angewandt: Zuerst werden die Listen in der Mitte geteilt und dann MergeSort auf die beiden Teillisten angewandt. Die Listen werden so lange zerstückelt, bis atomare Listen vorliegen. Diese sind sortiert. Dann werden die Listen mithilfe von Vergleichen sortiert zusammengefügt. Das passiert in allen Schritten wieder rückwärts und die Liste ist sortiert. Abbildung ?? zeigt eine Implementierung in Pseudo-Code.

```

1  int[] mergeSort(int[] liste)
2      if länge(liste).size == 1
3          return liste
4      else
5          int mitte = länge(liste)/2
6          int[] vordereListe = teilliste(liste, 0, mitte);
7          int[] hintereListe = teilliste(liste, mitte, länge(liste));
8          vereineListen(vordereListe, hintereListe)

```

Abbildung 2.4: MergeSort in Pseudo-Code

QuickSort

Die Idee ist, dass immer ein Element als Pivotelement gewählt wird. Oft wird das Element in der Mitte der Liste gewählt. Dann werden alle Elemente $\leq$ in die linke Teilliste gebracht und der rest in die rechte. Einelementige Listen sind bereits sortiert. Alle Teillisten werden wieder mit Quicksort sortiert und zusammengefügt. Abbildung ?? zeigt eine

Es wird nicht erklärt, wie ein Heap erstellt wird oder wie die Heapify operation funktioniert.

Befehle, wie teilliste(), vereineListen() erklären. Erklären, dass der index vorne inklusiv und der index hinten exklusiv ist.

Implementierung in Pseudo-Code, bei das Element in der Mitte als Pivotelement gewählt wird.

```

1  int[] quickSort(int[] liste)
2      if länge(liste).size == 1
3          return liste
4      else
5          int pivotelement = liste[länge(liste)/2]
6          int[] vordereListe = {}
7          int[] hintereListe = {}
8          for n in {0, ..., länge(liste)-1}
9              if(liste[n] <= pivotelement)
10                 anhängen(vordereListe, liste[n])
11             else
12                 anhängen(hintereListe, liste[n])
13         return konkateniere(vordereListe, hintereListe)

```

Abbildung 2.5: QuickSort in Pseudo-Code mit dem ersten Element als Pivotelement

Statt für das Pivotelement das mittlere Element zu wählen können auch andere Elemente gewählt werden. Besonders gute Ergebnisse in natürlich aufkommenden unsortierten Listen ergeben sich mit der **Median-Of-3**. Hierbei werden die Elemente am Anfang, in der Mitte und am Ende verglichen und es wird das Element welches zwischen den beiden anderen liegt als Pivotelement gewählt. Zum Beispiel wird bei den drei Werten $-39, 75, 940221$ die Zahl 75 als Pivotelement gewählt.

Für Wahlen des Pivot-Elements kann jedoch ein Worst-Case-Szenario gebaut werden. Die Laufzeit liegt dann bei $\mathcal{O}(n^2)$.

Ich mache hier ein Anhängen an ein Array. Dadurch wird die Laufzeit natürlich nicht erreicht.

HeapSort

Konzeptionell wird beim HeapSort die Liste in einen Heap umgewandelt. Das ist ein spezieller Binär Baum mit der Eigenschaft, dass die Elemente in einem Knoten größer sind als die Elemente in den Kindern. Der Baum wird Pre-Order-Traversierung als Liste dargestellt. Das größte Element, steht dann am Anfang der Liste. Wir vertauschen das erste und das letzte Element und wenden die Heapify-Operation auf der unsortierten Teilliste an, um den vorderen Teil wieder in einen Heap umzuwandeln. HeapSort kann als eine

```

1  int[] quickSort(int[] liste)
2      liste = erstelleHeap(liste)
3      for n in {länge(liste)-1, ..., 0}
4          liste = heapify(liste, 0, n+1)
5          tausche(liste[0], n)
6      return liste

```

Abbildung 2.6: HeapSort in Pseudo-Code mit dem ersten Element als Pivotelement

Verbesserung des SelectionSort gesehen werden, da beim HeapSort das größte Element aus einer Liste gesucht wird und an das Ende gesetzt wird.

Was ein Heap ist, kann erst im Baum-Teil richtig erklärt werden. Ist das in Ordnung? Oder sollen wir Datenstrukturen und Algorithmen trennen?

2.2.2 Suchalgorithmen

Lineare Suche

Die Suche wird auch als sequenzielle Suche bezeichnet. Es wird die Liste von vorne her durchgegangen bis das entsprechende Element gefunden ist.

Die Laufzeit liegt im Worst-Case in $\mathcal{O}(n)$, weil die Liste einmal durchgegangen werden muss.

Binäre Suche

Diese Form funktioniert nur auf sortierten Listen. Es wird zur Mitte gesprungen und verglichen, ob der Schlüssel kleiner, gleich oder größer ist. Dann wird entweder nach links oder rechts gesprungen oder das Element ausgegeben.

Bei jedem Schleifendurchlauf halbiert sich der zu durchsuchende Bereich, die Komplexität liegt also in $\mathcal{O}(\log(n))$. Sortieren davor erhöht die Laufzeit.

2.3 Bäume

2.3.1 Eigenschaften in Binärbäumen

Ein Binärbaum t der Höhe h hat die folgenden Eigenschaften:

- t hat maximal $2^{h+1} - 1$ Knoten.
- t hat mindestens $2^{h-1} + 1$ Knoten.
- t hat maximal $2^h - 1$ innere Knoten.
- t hat maximal 2^h Blätter

Ein Binärbaum kann in ein Array eingebettet werden. Dies wird erreicht durch das Einspeichern der Knoten von links nach rechts, gestaffelt in den Höhen. Also sind die Elemente aus der Höhe 2 (Es sind $2^{2-1} = 2$ Elemente) an den Arrayindizes 2 und 3 zu finden (An den Stellen 2^{h-1} bis $2^h - 1$).

2.3.2 Baumtraversierungen

Es werden üblicherweise drei Methoden verwendet:

- **Preorder:** Knoten - linker Teilbaum - rechter Teilbaum
- **Inorder:** linker Teilbaum - Knoten - rechter Teilbaum [**Mehrdeutigkeit möglich**]
- **Postorder:** linker Teilbaum - rechter Teilbaum - Knoten
- **Breitendurchlauf:** Ebenenweise durchlaufen (wie bei der Arrayeinbettung)

2.3.3 Binäre Suchbäume

Ein binärer Suchbaum erleichtert das Suchen. Ist der Knoten selbst nicht das gesuchte Element, so kann nach links gegangen werden um alle kleineren Elemente zu betrachten und nach rechts um alle größeren zu durchsuchen.

Beim **Einfügen** eines Schlüssels wird eine Suche nach dem Element gestartet und das Element dort eingefügt, an der die Suche nicht weiterkommt. Somit wird der neue Schlüssel ein Blatt.

Beim **Entfernen** eines Schlüssels wird dieser gesucht. Ist er nicht vorhanden, passiert nichts. Ist er ein Blatt, so wird dieses entfernt. Ist es ein innerer Knoten, so wird er mit dem rechtesten Blatt des linken Teilbaums vertauscht und dann entfernt.

Laufzeiten in binären Suchbäumen

Die Best-Case Laufzeit liegt bei $\mathcal{O}(\log(n))$ und im Worst-Case bei $\mathcal{O}(n)$. Das liegt daran, dass maximal die ganze Baumtiefe durchsucht werden muss. Das kann im schlimmsten Fall eine lineare Liste sein und im besten Fall ein ausbalancierter Baum.

2.3.4 Varianten von Binärbäumen

Für bessere Suchergebnisse werden noch zwei weitere Arten von Binärbäumen verwendet:

- **AVL-Bäume**
- **Max-Heap**
- **Splay-Bäume**
- **B-Bäume/Mehrwegeebäume**

Max-Heap

Max-Heaps sind Bäume mit der Eigenschaft, dass das Element im Elternknoten größer ist als die Elemente in den Kindknoten. Außerdem sind alle Ebenen bis auf die letzte vollständig aufgefüllt und die letzte Ebene ist von links aufgefüllt. **Min-Heaps** sind genauso definiert, nur dass das Element im Eltern-Knoten kleiner ist als die Elemente in den Kindknoten.

Intuitiv sind Heaps fast sortierte Listen, denn mittels HeapSort lassen sie sich besonders schnell sortieren.

Unsortierte Listen können in einen Heap umgewandelt werden, in dem die Heapify-Operation angewandt iterativ angewandt wird. Für einen gegebenen Baum, bei dem nur die Wurzel die Heapify Bedingung nicht erfüllt, vertauscht die Heapify die Wurzel mit einem seiner Kinder, sodass Elternelement größer ist als die Kindelemente. Da nun sein kann, dass ein Kind die Heap-Bedingung nicht erfüllt, wird dort Heapify rekursiv fortgeführt. Bei einer unsortierten Liste wird erst bei den Blättern die Heap-Bedingung sichergestellt. Nach oben gehend wird dann mit der Heapify-Operation der ganze Baum in einen Max-Heap umgewandelt.

Wir identifizieren wir Listen mit Bäumen. Das ist noch nicht behandelt worden.

AVL-Baum

AVL-Bäume sind Binärbäume, die die Struktureigenschaft haben, dass jeweils beide Teilbäume eines Knotens sich in der Höhe nur um maximal 1 unterscheiden. Damit werden immer schnelle Suchen garantiert.

Zeitaufwand entsteht dadurch, dass beim Einfügen und Entfernen von Elementen der Baum durch Rotationen neu balanciert werden muss. Der Gesamtaufwand einer Rotation liegt bei $\mathcal{O}(\log(n))$. Dabei ist höchstens eine (Doppel-)Rotation nötig. Betrachte dafür die Bilder auf Wikipedia.

Splay-Baum

Splay-Bäume sind selbstoptimierende Bäume. Nach einer Suche wird der Schlüssel an die Wurzel rotiert. Dabei ist das Laufzeitverhalten asymptotisch die eines optimalen Baums. Es wird auch weniger Speicher verbraucht, da entgegen dem AVL-Baum keine Daten über die Tiefe etc. gespeichert werden müssen. Die Laufzeit selber ist $\mathcal{O}(\log(n))$.

Splay-Bäume wurden noch nie im Stex abgefragt

B-Baum

Wenn der binäre Suchbaum auf einem externen Speicher geladen ist und nicht vollständig in den RAM geladen werden kann, dann kann selbst das Suchen in einem Baum ineffizient werden. An jedem Schritt muss der Inhalt des Knotens in den Arbeitsspeicher gebracht werden, damit er dort vom Prozessor mit dem gesuchten Wert verglichen werden kann. Die Höhe des Baumes trägt damit maßgeblich zur Laufzeit der Suche bei. Um die Höhe zu verringern, könnte man den Baum balancieren und zusätzlich könnte ein Knoten mehrere Werte und mehr als 2 Kinderbäume haben. Die Kinderbäume speichern die Werte, die zwischen den Knotenwerten liegen. Einen solchen Baum nennen wir **B-Baum**. Er findet besonders bei Datenbanksystemen Anwendung, wo mit großen Datenmengen umgegangen werden muss.

Ein **B-Baum** ist ein Baum mit folgenden Eigenschaften:

1. Jeder Knoten außer der Wurzel ist eine sortierte Liste mit mindestens $t - 1$ und maximal $2t - 1$ Schlüssel und zwischen diesen Verweise auf Kindknoten.
2. Die Wurzel ist eine sortierte Liste mit mindestens 1 und maximal $2t - 1$ Schlüssel und zwischen diesen Verweise auf Kindknoten.
3. Alle Blattknoten haben die gleiche Tiefe.
4. Sitzt ein Verweis auf einen Kind-Teilbaum zwischen zwei Schlüsseln x, y , dann sitzen alle Elemente des Teilbaums zwischen x, y

Wird ein neues Element zum Baum hinzugefügt, dann wird dieses immer einem Blatt hinzugefügt. Hat ein Blatt dadurch mehr Schlüssel als erlaubt, dann wird der Knoten am Median der Schlüssel aufgeteilt. Der Median wird dem Elternknoten hinzugefügt und die zwei Teilbäume werden zwei Kinder des Elternknoten. Hat der Elternknoten dadurch zu viele Elemente, wird das Vorgehen nach oben rekursiv wiederholt.

Wird ein Element entfernt und ein Knoten hat dadurch zu wenige Schlüssel, dann versuchen wir ein Element aus einem Geschwisterknoten zu nehmen. Um dabei die Sortierung des Baumes zu erhalten, machen wir das Mithilfe einer Rotation. Können wir kein Element aus einem Geschwisterknoten nehmen, weil die auch zu wenige Elemente haben, dann vereinen wir Kind und Geschwisterknoten. Hat jetzt der Elternknoten zu wenige Schlüssel, dann lösen wir das Problem rekursiv.

2.4 Graphen

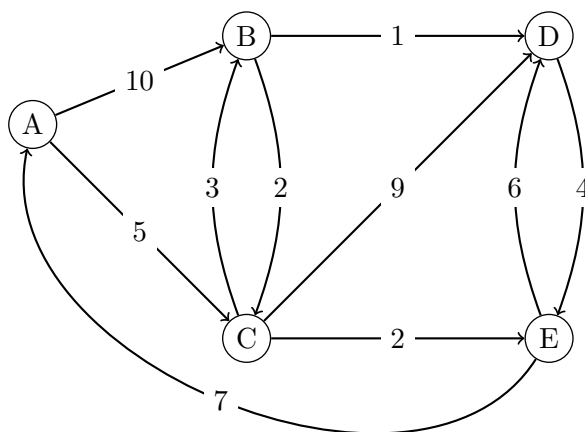
2.4.1 Algorithmus von Dijkstra

Will man den kürzesten Weg zwischen zwei Knoten A, Z in einem Graph finden, dann erhält man die Antwort durch den Algorithmus von Dijkstra.

Die Idee des Algorithmus ist, dass man die kürzesten Pfade aller Knoten zu A findet. Da wir die Entfernungen zu A am Anfang noch nicht wissen, initialisieren wir die Entfernung zu allen Knoten außer a mit ∞ . Der Algorithmus beginnt im Knoten A und führt dann iterativ zwei Schritte aus:

1. Berechne eine obere Schranke für Entfernungen
2. Von allen bisher unbesuchten Knoten, gehe zu dem Knoten mit der kleinsten Entfernung zu A

Eine beispielhafte Ausführung des Algorithmus ist in Abbildung ?? gegeben.



k	W_k	S_k	$D_k(B)$	$D_k(C)$	$D_k(D)$	$D_k(E)$
0	—	$\{A\}$	10	5	∞	∞
1	C	$\{A, C\}$	8	5	14	7
2	E	$\{A, C, E\}$	8	5	13	7
3	B	$\{A, C, E, B\}$	8	5	9	7
4	D	$\{A, C, E, B, D\}$	8	5	9	7

Abbildung 2.7: Beispiel des Dijkstra-Algorithmus

2.4.2 Bellman-Ford-Algorithmus

Der Bellman-Ford Algorithmus berechnet kürzeste Wege, erlaubt aber anders als Dijkstra auch negative Kantengewichte. Wir nehmen an, dass der Graph keine Zyklen enthält, deren Kantengewichte summiert eine negative Zahl ergeben.

Die zentrale Einsicht ist, dass ein kürzester Weg zwischen zwei Knoten v_1, v_2 in einem Graph $G = (V, E)$ maximal $|V| - 1$ Pfade enthalten kann. Andernfalls besucht er einen Knoten doppelt und enthält somit eine Schleife. Wir können also eine verbesserte Breitensuche durchführen und diese findet in $|V| - 1$ Schritten garantiert den kürzesten Pfad.

Starte mit einem Knoten v_1 . Initialisiere die Entfernung zu v_1 als 0 und zu anderen Knoten als ∞ . Führe folgenden Schritt $|V| - 1$ mal für jede Kante' aus: Folge der Kante e vom Knoten u zum Knoten v . Ist die Entfernung von v größer als die Entfernung von u plus dem Kantengewicht, dann aktualisiere die Entfernung von v entsprechend.

2.4.3 Minimale Spannbäume

Ein Minimaler Spannbaum ist ein ungerichteter Subgraph eines ungerichteten Graphen. Jeder Knoten ist enthalten und die Kantensumme ist minimal. Für die Berechnung gibt es zwei Methoden:

Prim-Algorithmus

Starte bei einem beliebigen Knoten. Dann wird von diesem Knoten ausgehend die minimale Kante hinzugefügt. Dieser Schritt wird immer für den gesamten Subgraph wiederholt. Finde von allen hinzugefügten Knoten die dann minimale Kante hinzu, die einen noch nicht erreichten Knoten hinzufügt. So bleibt die Zyklenfreiheit erhalten. Die Laufzeit liegt in $\mathcal{O}(|V|^2)$

Algorithmus von Kruskal

Zuerst werden die Kanten der Länge (des Gewichts) nach sortiert. Dann werden die Kanten schrittweise dem minimalen Spannbaum hinzugefügt. Eine Kante wird nicht hinzugefügt, falls der neu verbundene Knoten bereits im Spannbaum enthalten ist. Somit ist die Zyklenfreiheit und die minimale Kantensumme gewährleistet. Die Laufzeit liegt in $\mathcal{O}(|E| \cdot \log(|E|))$

2.5 Hashing

Ziel ist es, eine Schlüsselmenge auf eine (relativ) kleine Liste zu schreiben. Dafür benötigt man eine Hashfunktion. Diese muss zumindest surjektiv sein.

2.5.1 Umgang mit Kollisionen

Bei einer Kollision beim Einfügen müssen Lösungen der Kollision gefunden werden. Dabei wird in zwei Kategorien unterschieden:

- **Offenes Hashing mit geschlossener Adressierung:** Hier wird bei einer Kollision die Adresse mit einer Datenstruktur verknüpft, die eine Suchfunktion bietet. Je nach Methode wird auch direkt jeder Hashwert mit einer solchen Struktur verknüpft.
- **Geschlossenes Hashing mit offener Adressierung:** Hierbei wird bei der Kollision eine Sondierungsfunktion verwandt. Die Hashtabelle wird weiter durchgegangen und es wird bis zum nächsten freien Platz sondiert.

2.5.2 Gängige Sondierungen

- **Lineares Sondieren:** Hierbei wird nach einer Kollision der nächste freie Platz in c_1 Schritten gesucht:

$$h(x, j) = (h(x) + c_1 j) \mod m$$

- **Quadratisches Sondieren:** Hierbei wird nach einer Kollision in quadratischer Schrittweite ein neuer Platz gesucht:

$$h(x, j) = (h(x) + c_2 j^2)$$

- **Ein Versuch für optimales Sondieren:** Hierbei werden zwei Hashfunktionen h und h' verwandt, deren Kollisionswahrscheinlichkeit bei $\frac{1}{m}$ liegt (also dass $\mathbb{P}(h(x) = h(y)) = \frac{1}{m} = \mathbb{P}(h'(x) = h'(y))$ liegt, dabei sind die x in beiden Vorkommen gleich genauso wie die y). Die Sondierung lautet dann

$$h(x, k) = (h(x) + h'(x)j^2)$$

2.6 Algorithmische Paradigmen

2.6.1 Backtracking

Idee:

- Lösung eines Problems durch Versuch und Irrtum (trial and error) .
- Schrittweises „Herantasten“ an die Gesamtlösung.
- Kann eine Teillösung nicht zu einer Gesamtlösung erweitert werden:
 - Letzten Schritt rückgängig machen.
 - Weitere, alternative Schritte probieren.

Voraussetzung:

- Die Lösung setzt sich aus „Komponenten zusammen“
- Für jede Komponente gibt es mehrere Wahlmöglichkeiten
- Teillösungen können getestet werden.

Geeignete Probleme:

- Damenproblem
- Sudoku
- Solitär-Brettspiel
- Wegsuche von A nach B

2.6.2 Greedy-Algorithmen

Bei Greedy-Algorithmen wird aus dem aktuellen Lösungszustand immer der Nächste gewählt, der für den Moment der nächstbeste erscheint. Ein Beispiel dafür ist der Prim-Algorithmus.

Das Problem daran ist, dass diese Algorithmen nicht immer die optimale Lösung finden. Jedoch liefern sie meist eine schnelle und relativ gute Lösung.

2.6.3 Dynamische Programmierung

Die dynamische Programmierung versucht erst kleinere Probleme zu lösen. Diese Lösungen sollen dann eine Hilfe oder Grundlage für das nächstgrößere Problem sein.

Ein Beispiel kann die Berechnung der Fibonacci-Funktion sein. Ein klassischer rekursiver Algorithmus berechnet $fib(n)$ rekursiv aus der Summe von $fib(n-1)$ und $fib(n-2)$. Die Komplexität des Algorithmus ist damit $\Theta(2^n)$. Bei dynamischer Programmierung hingegen wird erst $fib(1)$, $fib(2)$, ... berechnet und daraus $fib(n)$ zusammengesetzt. Die Komplexität ist somit nur noch bei $\Theta(n)$. Das Knapsack-Problem lässt sich ebenfalls damit lösen.
<https://www.youtube.com/watch?v=cJ21moQpofY>.

2.7 Zusammenfassung

3 Softwaresysteme

3.1 Projektplanung

3.1.1 Softwaremaße [H16T1T2A1]

Um ein SW-Projekt zu bemessen gibt es verschiedene Arten zu messen. Dabei gibt es drei bekannte Maße:

- **Größenorientierte Maße (LOC/NCSS):** Hierbei wird die Anzahl der Codezeilen gemessen. Probleme: Wann arbeitet man viel? Wie wird die Qualität sichergestellt? Wie lang sind Anweisungen in verschiedenen Programmiersprachen?
- **Funktionsorientierte Maße:** Versuch, die Komplexität aus der Funktionalität abzuleiten. Wie viele verschiedene Funktionen hat eine Software. Problem: Sie ist stark abhängig von der Qualität des Entwurfs oder der Anforderungen und kann erst spät angeandt werden.
- **McCabe:** Sie wird auch **zyklomatische Komplexität** genannt. Dazu werden die Module auf ihre Komplexität hin untersucht. Dafür wird der Kontrollflussgraph gezeichnet und die Komplexität entspricht:

$$V(Z) = |E| - |V| + 2$$

Es fällt auf, dass beispielsweise Sequenzen die Komplexität nicht erhöhen. Neue Schleifen und If-Else-Bedingungen hingegen schon.

- **Objektorientierte Maße:** Ableitung der Komplexität aus der Objektstruktur. Probleme: Wie größenorientiert und kann bei vielen Sprachen nicht angewandt werden.

3.1.2 Qualitätskriterien von Code

- **Korrektheit**
- **Wartbarkeit**
- **Integrität:** Wahrscheinlichkeit einer erfolgreichen Attacke.
- **Benutzbarkeit**

3.1.3 GANTT-Diagramme [F15T1T1A3, H15T1T1A3, ...]

In diesen Diagrammen werden die Arbeitspakete mit Boxen eingetragen. Die Länge der Box entspricht der geschätzten Dauer. Alle Dependencies sind mit Pfeilen gekennzeichnet. Ein Paket kann frühestens eingetragen werden, wenn alle Dependencies gelöst sind.

Es werden so viele Mitarbeiter benötigt, wie es maximal gleichzeitig laufende Pakete gibt.

Die **kritischen Pfade** sind solche, die die gesamte Dauer eines Projekts umspannen. Ein Beispiel für ein GANTT-Diagramm gibt es [hier](#).

3.1.4 PERT/CPM Netzwerke [F15T1T1A3, H15T1T1A3, ...]

In einem CPM-Netzwerk werden die Anfänge oder Enden eines Projekt-Meilensteins verzeichnet. Die Kanten tragen die Dauer um die Aktivität zu machen. Zusätzlich werden in den Knoten die früheste und späteste Startzeit/Endzeit eingetragen. Dependencies ohne Zeitaufwand werden gestrichelt dargestellt.

doubt

Um die früheste Startzeit für jeden Meilensteing zu berechnen, werden die Abhängigkeiten und benötigten Zeiten von vorne durchgerechnet. Um die späteste zu berechnen, werden die benötigten Zeiten von hinten durchgerechnet. Dadurch erkennt man kritische Pfade. Knoten auf einem kritischen Pfad sind solche, die eine gleiche früheste und späteste Startzeit haben.

3.2 Testen

Beim Testing gibt es zwei grundsätzlich unterschiedliche Verfahren:

- **White-Box Testing:** Die Testfälle werden anhand des vorliegenden Codes generiert und durchgeführt.
- **Black-Box Testing:** Es ist nur bekannt was der zu testende Code machen soll und welche Eingaben er annehmen soll. Aufgrund dieser Wissenslage werden dann Testfälle generiert.

3.2.1 Kontrollflussgraphen (CFG) [H17T1T1A4, H17T1T2A4, ...]

Ein Kontrollflussgraph ist eine Repräsentation der möglichen Wege in einem Programm (-abschnitt). Dabei sind die Codezeilen Knoten und die Anweisungen sind die Kanten. Die Zyklen in einem CFG entsprechen den Schleifen in einem Programm.

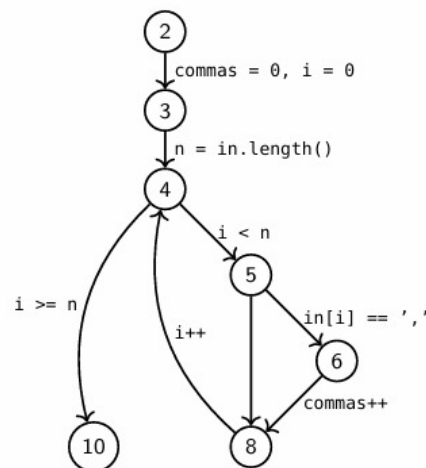
Das Beispiel stammt aus der Vorlesung FSV.

Programm

```

1  int countCommas(String in) {
2      int commas = 0, i = 0
3      int n = in.length();
4      while(i < n) {
5          if(in.charAt(i) == ',') {
6              commas++;
7          }
8          i++;
9      }
10     return commas ;
11 }
```

Kontrollflussautomat



3.2.2 Überdeckungsmaße [H14T2T2A3, H15T2T1A2, ...]

Anhand des Kontrollflussgraphen lassen sich mehrere Qualitätsmaße eines Tests definieren.

- **statement coverage / Anweisungsüberdeckung:** Wurden alle Knoten/Anweisungen einmal besucht?
- **branch coverage / Zweigüberdeckung:** Wurden alle Fallunterscheidungen genommen?
- **path coverage / Pfadüberdeckung:** Wurden alle möglichen Pfade durch das Programm ausgeführt (meistens unendlich viele...daher nicht so geeignet)

3.3 Sequentielle Prozessmodelle

3.3.1 Wasserfallmodell [H14T2T1A2, H20T2T1A1, ...]

Das Wasserfallmodell beschreibt eine mögliche Herangehensweise an ein Softwareprojekt. Es läuft ab in 5 Schritten:

1. Anforderungsdefinition (Lasten- und Pflichtenheft)
2. Entwurf (Entwurfsspezifikation)
3. Implementation (Programmcode und JavaDoc)
4. Testing (Testdokumentation und Berichte)
5. Wartung (Handbuch)

Vorteile des Wasserfallmodells sind, dass Festpreisverträge möglich sind. Zusätzlich kann der Fortschritt des Projekts leicht überwacht werden, es ist (sofern keine Probleme auftauchen) sehr effizient und kann leicht verstanden werden.

Die **Nachteile** des Wasserfallmodells sind, dass es häufig sehr lange Lastenhefte gibt, da alle Funktionalitäten nach dem ersten Schritt eingefroren sind. Die abstrakte Beschreibung des anfangs gewünschten Produkts ist meist für den Kunden sehr schwierig. Außerdem wird bei Kürzungen im Budget oder bei einer Verlängerung eventuell nichts ausgeliefert und der Kunde weiß erst am Ende, was er überhaupt für eine Software bekommt. Das Lernen findet erst nach dem Projekt statt.

3.3.2 V-Modell [H14T2T1A1]

Das V-Modell ist auch ein sequentielles Modell. Es setzt sich zusammen aus einer Planungs- und Entwurfsphase und aus einer Test- und Integrationsphase. Die Idee ist hierbei, dass man in der ersten Phase erst große und dann kleinen Systemteile entwirft. In der zweiten Phase werden erst kleine und dann große Systemteile integriert. Die einzelnen Bestandteile der beiden Phasen sind dabei einander gegenübergestellt.

Aus der Abbildung ist das Testen nicht ersichtlich. Vielleicht sollte man eine andere finden.

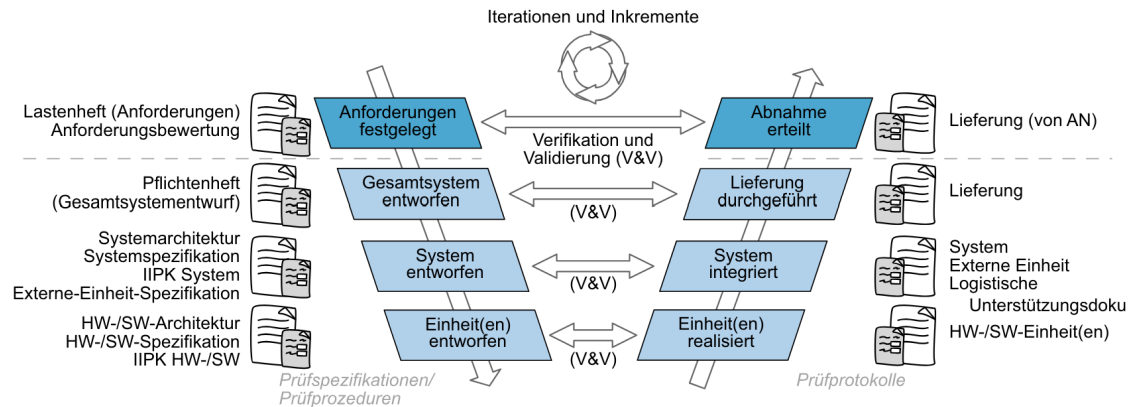


Abbildung 3.1: Das V-Modell

3.4 Agile Prozessmodelle

3.4.1 Inkrementelle und iterative Softwareentwicklung [H14T2T1A2]

3.4.2 Evolutionäre Softwareentwicklung [H16T1T2A1]

Hierbei wächst das System von einer kleinen Aufgabe zu einem immer größeren System heran. Die Schritte entscheidet eher der Kunde. Dabei ist eine Methode recht bekannt: **Prototyping**.

Das Vertikale Prototyping hat zum Ziel einen Durchstich durch das Produkt zu erstellen. Ein funktionaler Ausschnitt des Systems soll durch alle Ebenen hindurch entwickelt sein.

Das Horizontale Prototyping hat zum Ziel eine Ebene des Produkts vollständig zu entwickeln.

Dabei sind die Vorteile, dass die Kommunikation der Anforderungen leicht ist. Ansonsten analog zum Wasserfallmodell. Die Nachteile sind ebenfalls die des Wasserfallmodells und zum Beispiel, dass Kosten meist falsch eingeschätzt werden und die Anforderungen und Ausführung des Prototypen sehr schlampig sein können.

3.4.3 Spiralmodell [16T2T2A1]

Das Spiralmodell ist eine Spezialisierung des iterativen Modells und ein Vorläufer späterer Methoden wie SCRUM und dem EXtreme Programming. Es werden vier Quadranten spiralförmig durchlaufen:

1. Festlegung von Zielen, Alternativen, Rahmenbedingungen
2. Evaluieren von Alternativen und Risikoabschätzung
3. Realisierung des Zwischenzyklus
4. Planung des nächsten Zyklus

3.4.4 Methoden und Praktiken [F13T1T1A2, H13T1T2A3, ...]

Ein kleiner Überblick über die möglichen Methoden in agilen Projekten:

- Pair-Programming
- Test-Driven-Development
- Refactoring (möglich für Refactoring: Lesbarkeit, Erweiterbarkeit, Testbarkeit, Entfernung toten Codes, ...). Siehe auch Abschnitt ??.
- DevOps (Experten und Programmierer arbeiten zusammen, da Entwicklung und Operation zusammenwirken müssen)

3.4.5 SCRUM [H14T2T1A1, H20T1T1A5, ...]

Artefakte

Die drei Artefakte des SCRUM sind:

- Der **Product-Backlog** enthält alle Features die eine Software haben soll. Er wird vom Productowner verwaltet.
- Der **Sprint-Backlog** enthält alle Features, die innerhalb eines Sprints fertig gestellt werden sollen. Er wird durch das Entwicklungsteam am Anfang eines Sprints befüllt.
- Das **Increment** ist das Produkt, das zum aktuellen Entwicklungsstand ausgeliefert werden kann.

Rollen

- Der **Produktinhaber** ist der Auftraggeber und er entscheidet welche Features und in welcher Reihenfolge diese entwickelt werden. Er beantwortet Fragen der Entwickler und hat die Aufsicht.
- Das **Entwicklungsteam** ist für die Software, die Dokumentation und die Tests zuständig. Sie sind selbstorganisiert.
- Der **SCRUM-Master** kommuniziert die Werte, Prinzipien und Praktiken und ist verantwortlich für den Prozess. Er passt die Prozesse an das Produkt an. Er ist nicht der Projektmanager oder Personaler.

Ablauf

Ein Projekt das mit SCRUM gemanaged wird läuft in drei Phasen ab:

1. Überblicksplanung
2. Sprints
3. Projektabschluss

3.4.6 EXtreme Programming

3.4.7 Das agile Manifest [T21T2T1A2]

Das agile Manifest entstand als Folge der Verbreitung von SCRUM und EXtreme Programming. Es kennt 4 Leitsätze und 12 Prinzipien. Die Leitsätze lauten:

1. Individuum vor Prozess

2. Funktion vor Dokumentation
3. Mehr Zusammenarbeit mit dem Kunden als Vertragsverhandlungen
4. Reagieren auf Veränderungen als Festhalten an Plänen

Die Prinzipien lauten:

1. Frühe und kontinuierliche Auslieferung
2. Veränderungen sind immer (auch spät) willkommen
3. Funktionsfähige Produkte werden ausgeliefert
4. Experten und Entwickler arbeiten täglich zusammen
5. Individuen müssen motiviert sein
6. Gespräche sind das A&O
7. Das Fortschrittsmaß sind funktionierende Produkte
8. Gleiches Tempo auf unebgrenzte Zeit haltbar
9. Gutes Design und technische Exzellenz
10. Einfachheit
11. selbstorganisierte Teams
12. Reflektionen in den Teams

Die vier Leitsätze sind mal abgefragt worden. Die Prinzipien nicht. Vielleicht können wir sie entfernen.

3.5 Designpatterns [H16T2T2A1, H19T2T1A1, ...]

Architekturpattern wichtig fürs Systemdesign (wie ist das System aufgebaut? Zum Beispiel MVC, Server-Client (evtl mit API, Beschreibung welche Endpunkte hat der Server-/Backend)), Objektpatterns betreffen Abschnitte im Code (Unterkategorien: Behavioral (Observer, Template Method), structural (Singleton, Composite, Decorator), creational (factory)).

3.5.1 Abstract and Simple Factory [H16T1T2A2]

3.5.2 Iterator [H16T1T2A2]

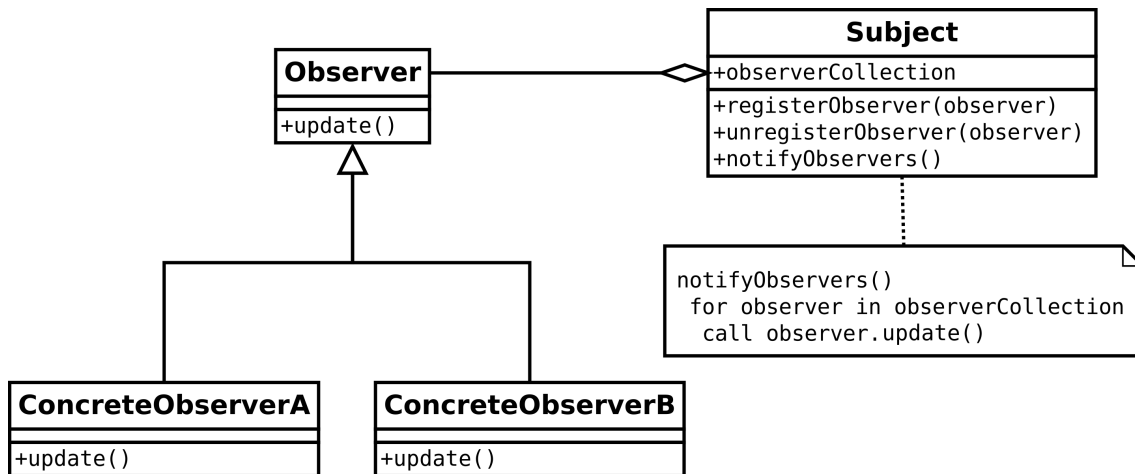
3.5.3 Decorator / Dekorierer [H16T1T2A2]

3.5.4 State / Zustand [H19T1T1A4]

3.5.5 MVC [H14T2T2A2]

Das Pattern Model-View-Controller ist ein Strukturmuster/Architekturmuster. Es gibt die drei Komponenten: Model, View und Controller. Der Controller verwaltet View und Model. Er ist auch für die Interaktion mit dem Benutzer zuständig. Dabei verändert der Controller die Daten im Modell und erhält per Observer die Änderungen des Benutzers im View. Per Observer erhält der View updates aus dem Model. Der View selber ist ein Kompositum

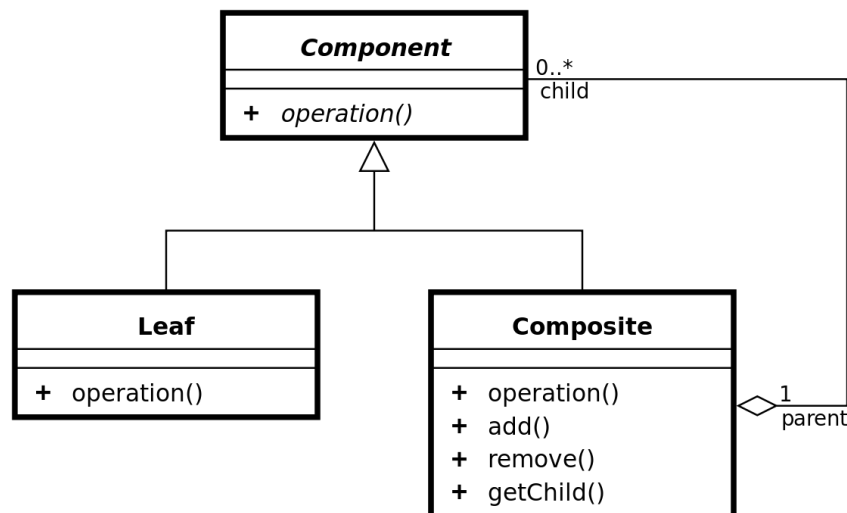
3.5.6 Observer [F18T2T2A2]



Das Observer-Pattern ist ein Verhaltensmuster. Dabei kann ein Subjekt mehrere Observer halten. Das Subjekt kann neue Observer registrieren und alte abmelden. Dabei ist die Observer-Klasse abstrakt und die speziellen Observer extenden Observer.

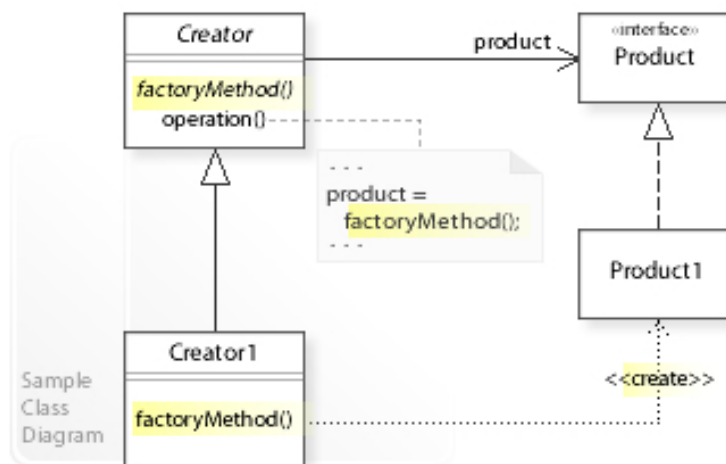
Sobald sich etwas am Subjekt ändert, können alle Observer notified werden. Diese handeln dann auf ihre eigene Weise.

3.5.7 Composite / Kompositum [H19T2T1A1]



Das Kompositum ist ein Strukturpattern und ist eine Liste von Objekten bei der jedes Objekt ein Kind des selben Typs hat und darauf zeigt. Das nennt sich einfach verkettet. Kennt jedes Objekt seinen Vorgänger und Nachfolger, so ist die Liste doppelt verkettet. Zusätzlich kann die Liste einen Start- und/oder einen Endknoten haben, die dann spezielle Varianten des eigentlichen Listenobjekts sind. Die Liste kann zusätzlich auch von einer Oberklasse verwaltet werden.

3.5.8 Factory [F15T1T2A3]

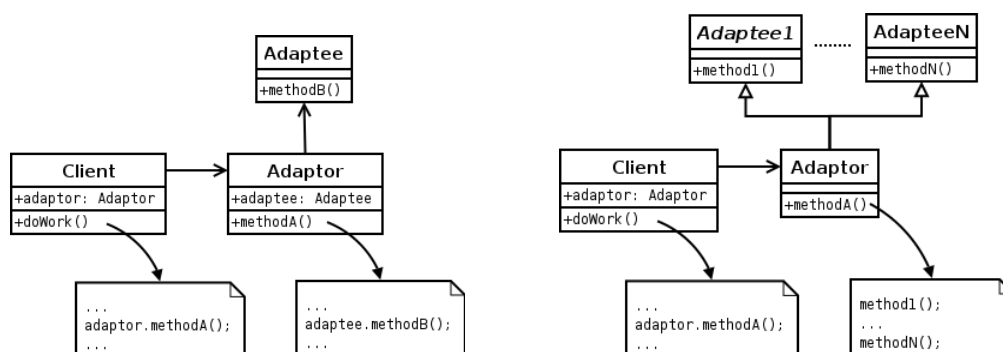


Abstract Factory stellt ein Interface zur Verfügung. Darin steht, was alles erstellt werden kann. (z.B. Stuhl, Sofa, Kaffeetisch). Aber das ist nur die Schnittstelle. Diese hat dann verschiedene Implementierungen: Die Viktorianische Fabrik, die moderne Fabrik und die ArtDeco-Fabrik. Alle drei produzieren jeweils Stuhl, Sofa, Tisch.

Das Factory Pattern ist ein creational pattern. Das bedeutet, dass es um die Erzeugung von Objekten einer Klasse geht. Dabei wird die Frage wie ein bestimmtes Objekt erzeugt wird den jeweiligen extends Klassen der Factory überlassen.

Ein Beispiel: In einem Spiel gibt es Räume (abstrakte Klasse). Das sind entweder normale oder magische Räume. Das bedeutet die beiden Klassen normaler und magischer Raum extenden Raum. Die abstrakte Klasse Game stellt die abstrakte Methode *makeRoom* zur Verfügung. Diese wird dann von den extendenden Klassen MagicGame und NormalGame erst mit Sinn befüllt. Daher wird je nach Struktur eine andere Art von Räumen erzeugt (eben kontextabhängig).

3.5.9 Adapter [F15T1R2A3, F16T1T2A2]



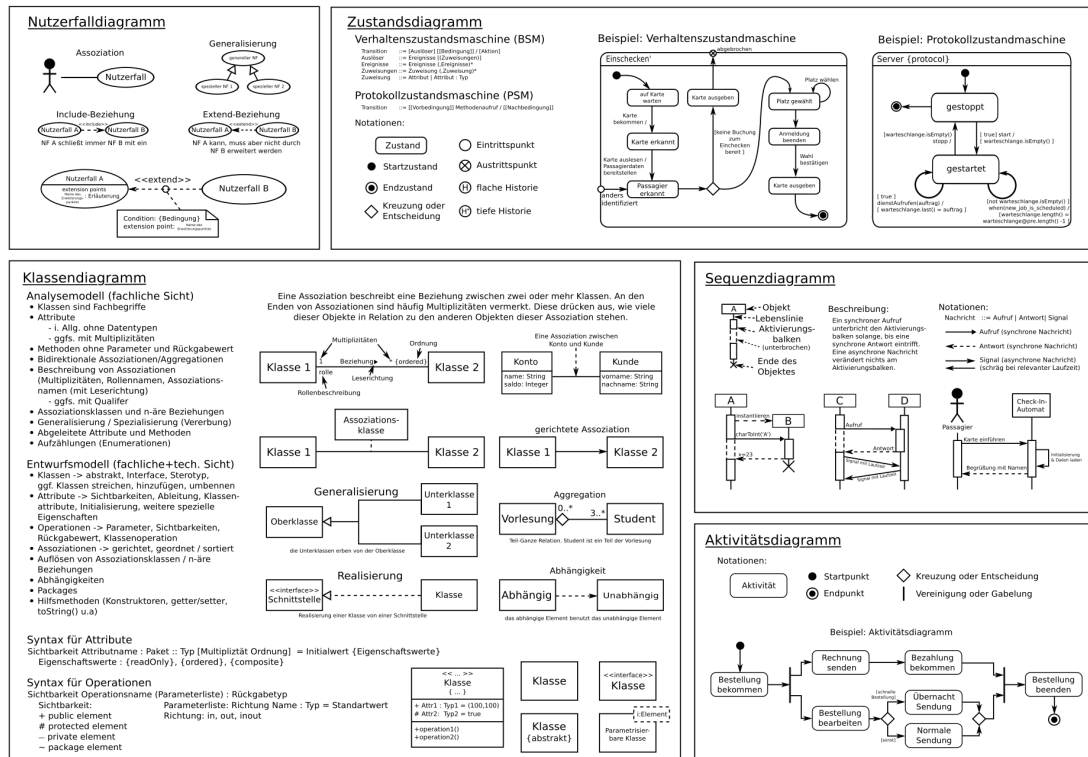
Der Adapter ist ein Strukturpattern und dient als Schnittstelle/Übersetzer zwischen zwei Klassen/Bibliotheken. Links ist der ObjectAdapter und rechts der ClassAdapter.

3.5.10 Singleton [H16T2T2A1, H20T1T1A3]

Ein Singleton ist eine Klasse die *final* ist. Es gibt immer nur maximal ein Objekt dieser Klasse. Wird versucht ein neues zu erzeugen, so erhält man das bereits existente zurück.

3.6 UML-Diagramme

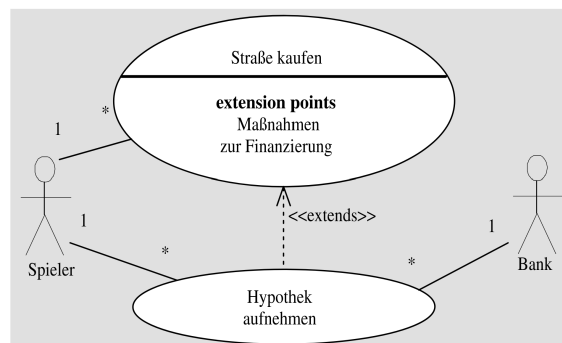
Der ultimative UML Überblick:



3.6.1 Use-Case-Diagramme [H14T2T1A3, F17T1R1A2, ...]

Wichtig für den ersten Schritt im V-Modell (Requirement analysis). Strukturierte Darstellung von Funktionalität (Kein Zusammenhang von Klassen). Wie hängen Funktionen für den Nutzer zusammen. Includes sind garantiert notwendige Bestandteile, Extensions sind Ausnahmen vom Regelfall.

Im Use-Case-Diagramm geht es um die Darstellungen von Handlungen und Akteuren. **Aktoren** können Menschen oder andere Systeme und Geräte sein. **Anwendungsfälle** werden durch Blasen dargestellt. Der Titel sollte direkt beschreiben, was hier passiert.



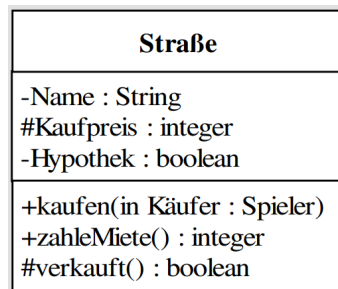
Jeder Anwendungsfall ist mit mindestens einem Akteur assoziiert. Diese Assoziationen

können auch verschiedene Wertigkeiten haben, wie das Beispiel gut zeigt (1 Spieler kann viele Straßen kaufen). Einige Anwendungsfälle müssen genauer beschrieben werden. Das ist durch die folgenden 3 Möglichkeiten realisierbar:

- Mit einem gestrichelten Pfeil und dem Schlüsselwort «*extends*» kann ein Anwendungsfall einen anderen erweitern. Dabei muss die Erweiterung nicht zwangsweise bei jeder Handlung auch verwendet werden. Sie ist optional. Die verschiedenen Erweiterungen heißen **Extension Points** und beschreiben die Erweiterungen. In dem Beispiel gibt es viele Methoden für eine Finanzierung der Straße. Hypothek ist dann eine davon. Es wären aber auch andere möglich.
- Mit einem gestrichelten Pfeil und dem Schlüsselwort «*includes*» kann ein Anwendungsfall einen anderen als grundsätzlich notwendigen Unterfall haben. Ein Beispiel: Bei einem Geldautomaten können verschiedene Aktionen vorgenommen werden. Aber sie alle beinhalten **zwangsweise** die Eingabe der PIN.
- Eine oben beschriebene Erweiterung oder Subroutine kann auch an Bedingungen geknüpft sein. Diese heißen **Conditions**. Sie werden durch Kreisen auf den Erweiterungspfeilen angedeutet. Dabei ist ein Post-it per gestrichelter Linie mit dem Kreis verbunden.

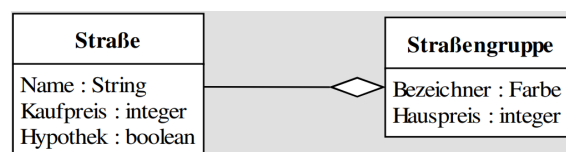
3.6.2 Klassendiagramme [H19T2T1A1, F21T1T1A3, ...]

Die Klassenkarte gibt Informationen über die Attribute und Funktionen der Klasse. Sichtbarkeiten werden wie folgt dargestellt: (+) *public*, (-) *private* und (#) *protected*.



Soll ein Objekt einer Klasse dargestellt werden, so ist der Name unterstrichen. Eine abstrakte Klasse oder ein Interface wird durch kursiv schreiben des Namens angezeigt.

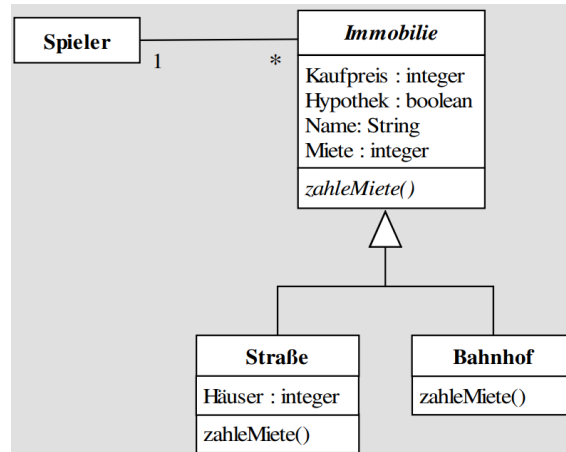
Die Beziehungen werden mit Strichen ausgedrückt. Dabei kann es noch Beschreibungen der Beziehungen geben: Zahlen am Ende einer Linie zeigen an, wie viele Objekte miteinander in Beziehung stehen. Zusätzlich kann Text auf dem Strich die Beziehung weiter beschreiben. Muss eine Beziehung durch eine weitere Klasse beschrieben werden, so wird die Klasse mit gestrichelter Linie an die Relation angehängt. Ist die Abhängigkeit gestrichelt, dann gibt es irgendeinen Bezug, aber man weiß nicht genauer wie es aussieht.



Die Aggregation wird mithilfe einer weißen Raute angezeigt. Sie beschreibt, dass eine Klasse zu ihrem Aggregat gehört ohne von ihm etwas wissen zu müssen. Eine Straße ist

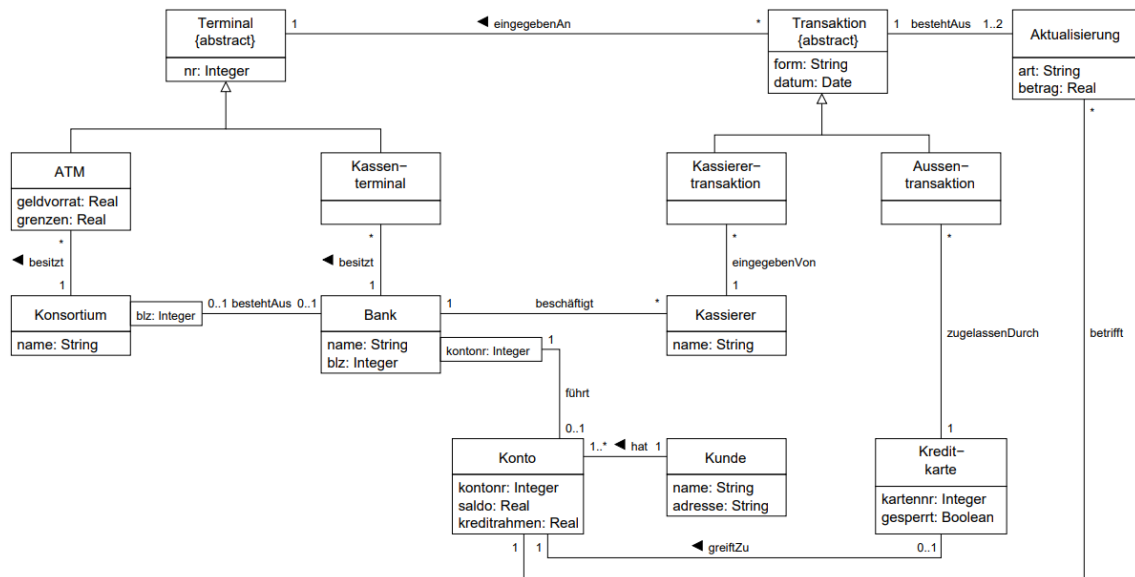
eben Teil einer Farbgruppe aber muss über die Gruppe selber nichts wissen.

Eine schwarze Raute ist die Darstellung einer Komposition. Sie ist die strengere Form der Aggregation. Hierbei kann die Unterklasse nicht ohne ihr Aggregat und andersherum existieren. Somit hat dort die Unterklasse den Status eines Attributs, kann aber nach der Erzeugung der Eltern erzeugt und vorher zerstört werden. Also ein Spiel besteht beispielsweise aus 40 Straßen und kann ohne diese nicht sinnvoll existieren.



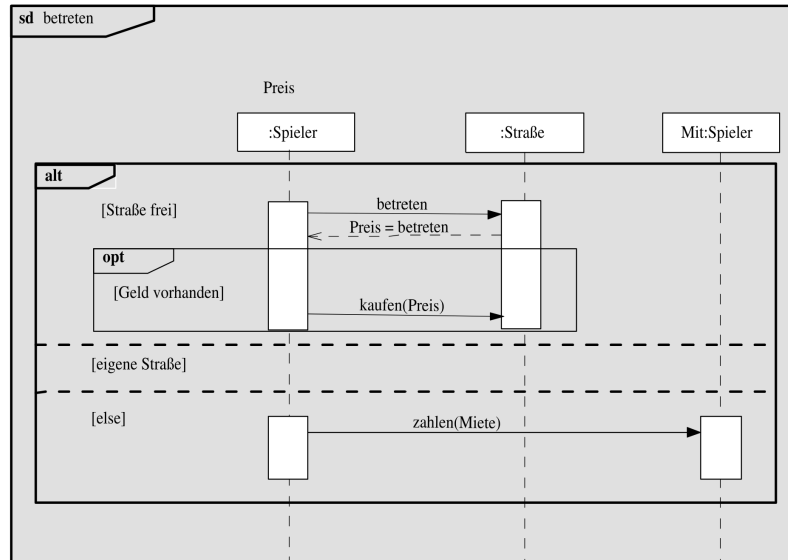
Eine Vererbung wird im Klassendiagramm mit Pfeilen mit weißen Spitzen dargestellt. Handelt es sich um eine abstrakte Klasse als Oberklasse, so sind die Methoden kursiv geschrieben. Handelt es sich um ein Interface, so wird der Pfeil gestrichelt gezeichnet und über dem Klassennamen *«interface»* vermerkt.

Ein vollständiges Klassendiagramm zeigt das Beispiel aus der Vorlesung:



3.6.3 Sequenzdiagramme [F13T2T1A1, H16T2T2A1, ...]

Aus der Vorlesung sind folgende ausführliche Sequenzdiagramm bekannt:



3.6.4 Zustandsdiagramme [H16T2T2A1, H18T1T1A3, ...]

Im Zustandsdiagramm werden die Zustände eines Systems modelliert. Dabei ist die Struktur ähnlich den Automaten. Die Übergänge der Zustände werden auf den Pfeilen beschrieben. Optionen ergeben sich durch Rauten. Verfeinerungen einzelner Zustände und Übergänge werden mit *ad* markiert.

3.7 Allgemeine Reste

3.7.1 User Stories

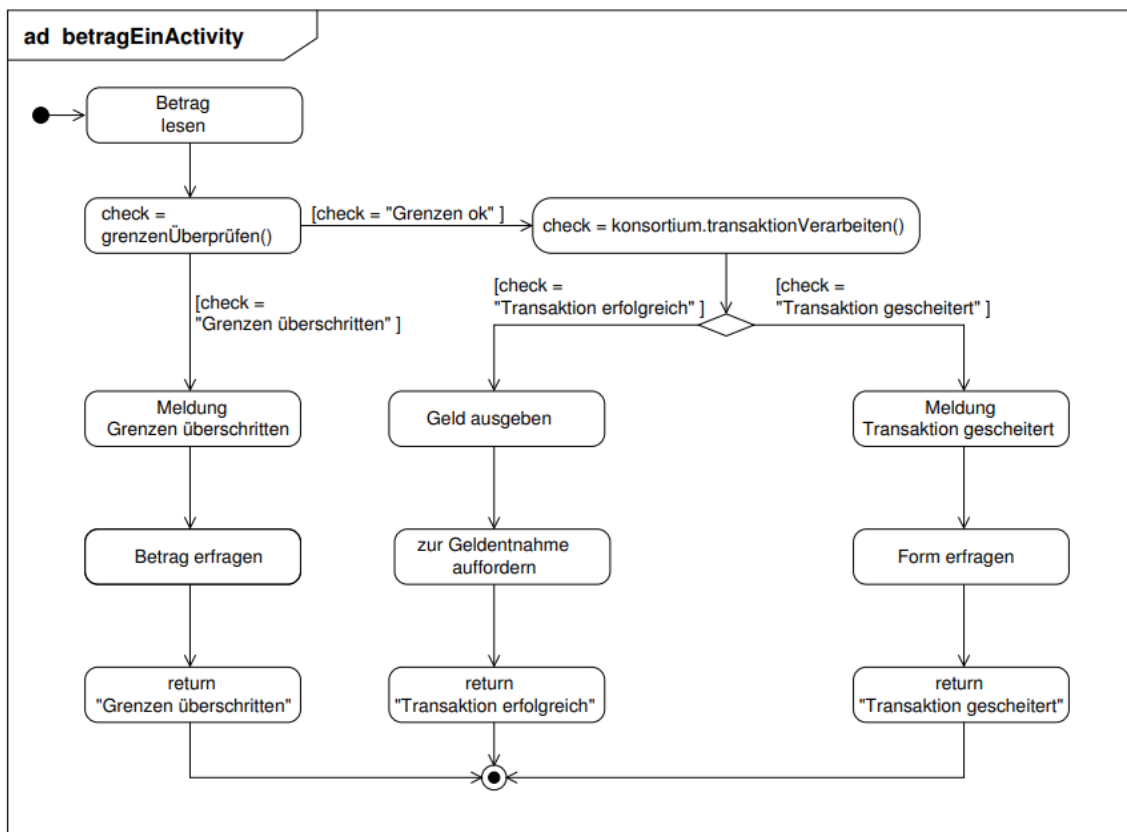
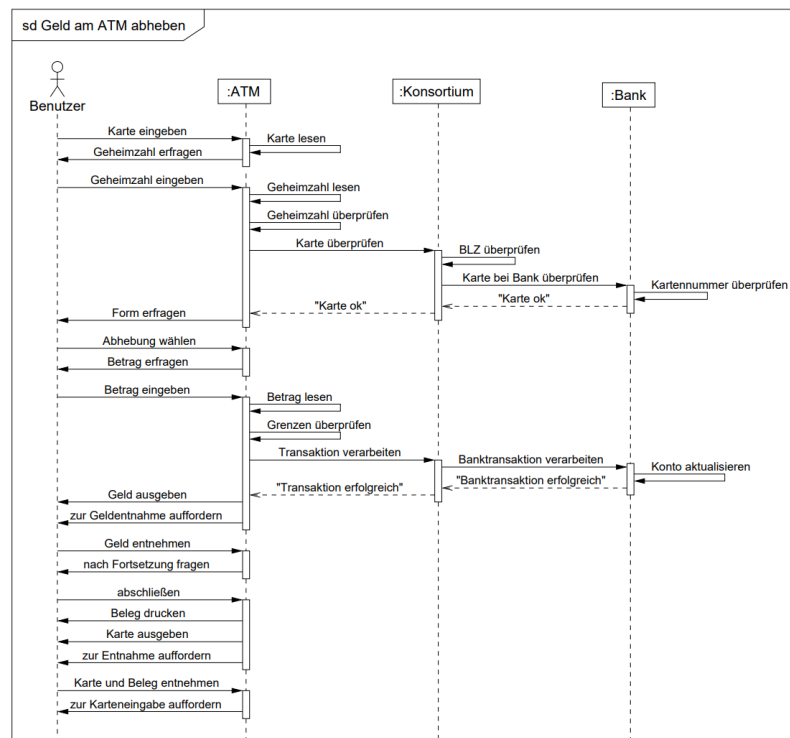
Eine User Story beantwortet die folgende Frage: As **WHO**, do I want **WHAT**, so that **WHY**. Auf der Vorderseite einer User Story steht dann die Beschreibung dieses Problems, also: Wer, was, warum? Auf der Rückseite einer User Story sind die Akzeptanzkriterien gelistet. Diese können dann aber auch deutlich umfangreicher sein, als der User das erwarten würde. Das ist aber dann die Seite für die Entwickler.

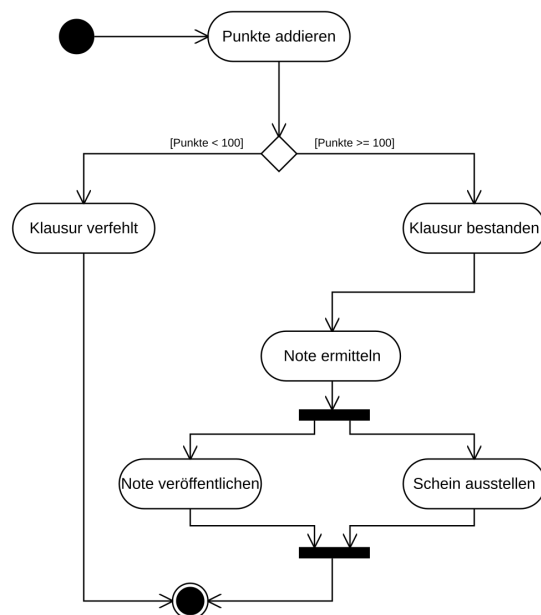
Beispiel: Als Student will ich meine Noten einsehen. So kann ich erfahren, ob ich die Klausur bestanden habe. Auf der Rückseite steht dann: Ein Student kann nur seine Noten einsehen. Die Noten können nicht manuell verändert werden. Und so weiter ...

3.7.2 Refactoring

Beim Refactoring geht es darum, den Code intern zu verbessern, so dass sich an seiner Verhaltensweise nichts ändert. Beziehungsweise nur soweit ändert, dass immer noch die gleichen Anforderungen realisiert bleiben. Zweiteres ist schwer abzugrenzen von einer Neuentwicklung oder einem Neuentwurf.

Ziel ist es, sogenannte Code-Smells los zu werden. Das kann über viele Arten geschehen: Prozeduren abstrahieren, neue Benennungen einführen, Kommentare entfernen/ergänzen, neue Klassen hinzufügen, ungenutzten Code entfernen, zyklomatische Komplexität reduzieren, etc ...





4 Datenbanken

MIN-MAX-Notation, Charakterisierung des schwachen Entitätstypen, Vor-Nachteile eines relationalen Systems, Relationen zusammenfassen/verfeinern, SQL in relationale Algebra kanonisch übersetzen, Dekompositionsalgorithmus, Zyklen in ER-Diagrammen, referentielle Integrität, Datenunabhängigkeit, Trigger, physische Optimierung,

4.1 Das relationale Modell

Im relationalen Modell werden alle Informationen in Form von Tabellen gespeichert, die sich einander referenzieren.

4.1.1 Begriffe

- Als **Relation** wird eine Tabelle verstanden.
- Eine **Datenbank** besteht aus einer Menge von Relationen/Tabellen.
- Ein **Tupel** ist eine Zeile einer Tabelle.
- Als **Attribute** werden die Spalten einer Tabelle bezeichnet.
- Die **Attributwerte** sind die Zellen der Tabelle.
- Als **Kardinalität** wird die Menge der Einträge einer Relation bezeichnet. Also die Einträge einer BD oder einer Abfrage.
- Die **Stelligkeit** einer Relation ist die Anzahl der verknüpften Domains.
- Das **Schema** bezeichnet die Anordnung und die vorhandenen Attribute (Spalten in einer Abfrage).
- Die **Ausprägung** ist dann die Liste der Elemente, die in der Relation enthalten sind.

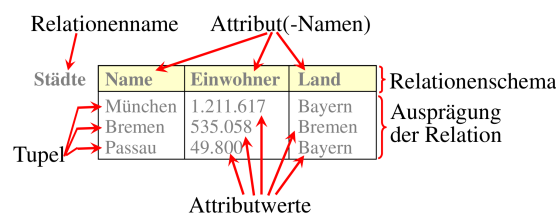


Abbildung 4.1: Aus DBS WiSe 2020: Die Grundbegriffe

4.1.2 Schlüssel [H15T1T2A1]

Eine Datenbank ist eine Sammlung von Relationen. Um die Relationen miteinander in Verbindung zu setzen, nutzt man Schlüssel. Der **Schlüssel** ist formal eine Menge von Attributen, die (1) jedes Element einer Tabelle eindeutig charakterisiert und (2) minimal ist. D.h. dass es keine Teilmenge von Attributen gibt, die Elemente eindeutig charakterisiert. Besteht die Menge aus mehreren Attributen spricht man von einem **zusammengesetzten Schlüssel**. Ein Attribut eines zusammengesetzten Schlüssels wird **prim** genannt. Eine Menge von Attributen, die nur Eigenschaft (1) erfüllt, wird **Superschlüssel**.

In der Praxis spricht man bei den möglichen Schlüsseln für eine Relation von **Schlüsselkandidaten** und wählt einen dieser Kandidaten als **Primärschlüssel** aus (meistens handelt es sich um eine ID). Prinzipiell erlauben aber Datenbanksysteme auch, aus Superschlüsseln einen Primärschlüssel zu machen. In dem Fall wird Schlüsselkandidat austauschbar für die Eindeutigkeitsbedingung (1) genutzt. Das ist aber schlechtes Datenbankdesign (siehe Kapitel zur Normalisierung).

- Der **Primärschlüssel** ist eine Wahl eines Schlüssels um Tupel in einer Relation eindeutig identifizieren zu können. Beachte, dass es Schlüssel geben kann, die keine Primärschlüssel sind.
- Ein **Fremdschlüssel** ist ein Attribut, das auf den Primärschlüssel einer anderen Relation zeigt.

4.1.3 Vermischte Informationen [H19T2T2A6]

- Unter der **Union Compatibility** versteht man das Prinzip, dass zwei Relationen mithilfe des Vereinigungsoperators $\cup$ verbunden werden können. Das bedeutet insbesondere, dass beide Relationen die selbe Anzahl Attribute und vom selben Datentyp haben müssen.
- Die Datentypen **BLOB** und **CLOB** sind Datentypen mit großer Kapazität (Binary/Character Large Object). Sie sind insbesondere dann von Interesse, wenn eine Datenbank viele XML Objekte speichern soll.
- **WAL** steht für Write-Ahead-Logging. Hierbei werden Änderungen in situ vorgenommen. Das heißt, dass vor allem Indexstrukturen nicht betroffen sind. Die Protokolle der Änderungen werden dann sequentiell gespeichert, was dazu führt, dass tatsächlich bis zu einem beliebigen Punkt zurück oder vorgesprungen werden kann. Das ist nützlich um für ACID die Consistency hoch zu halten.

4.2 Die 3-Ebenen-Architektur eines Datenbanksystems [F19T2T1A1]

Die typische Datenbank besteht aus drei Ebenen. Die wichtigste Ebene ist die **konzeptionelle Ebene**. Diese Ebene enthält die Modellierung der Realität als sogenanntes **logisches Schema**. Dieses beschreibt genau, wie verschiedene Daten miteinander in Zusammenhang stehen, beschreibt aber nicht, wie sie gespeichert werden. Das Schema wird mit der Data Definition Language (DDL) spezifiziert (Siehe Abschnitt ??).

Unter der konzeptionellen Ebene befindet sich die **interne Ebene**. Die Daten der Datenbank werden auf dem Computer gespeichert. Das logische Schema wird als **internes**

Schema implementiert. Hier wird beschrieben, wie die Daten gespeichert werden. Die exakte Umsetzung der internen Ebene ist abhängig von der Hardware und vom Betriebssystem.

Über der konzeptionellen Ebene ist die **externe Ebene**. Dabei handelt es sich um verschiedene Sichten, die verschiedenen Benutzergruppen zur Verfügung gestellt werden. Zum Beispiel könnten Händler und Kunden Zugriff zwar die gleiche Datenbank nutzen, aber durch Sichten verschiedenen Zugriff haben.

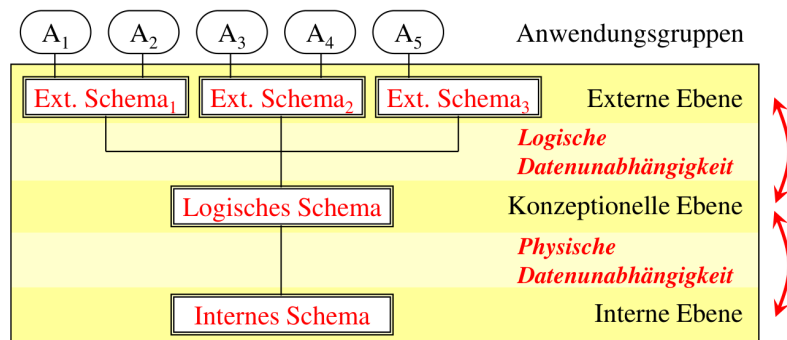


Abbildung 4.2: Ebenen eines Datenbanksystems nach ANSI/SPARC (Vorlesung Seidl)

Durch die Trennung der Datenbank in drei Ebenen und durch eine klare Spezifikation von Schnittstellen zwischen diesen Ebenen, erhält man zwei Typen von **Datenunabhängigkeit**:

- **Physische Datenunabhängigkeit:** Wird die Speicherung der Daten verändert (Veränderung des internen Schemas), so beeinflusst das nicht die konzeptionellen Ebene.
- **Logische Datenunabhängigkeit:** Werden die Sichten der Datenbank verändert, so beeinflusst das nicht die konzeptionelle Ebene.

4.3 DDL [F12T1T1A3, F19T2T1A5]

Was ist DDL?

```
create table Haelt
(
  Dozent    integer    references Dozenten (DNr)
              on delete cascade,
  VNr       integer    not null,
  Semester  varchar(20) not null,
  primary key (Dozent, VNr, Semester),
  foreign key (VNr, Semester) references Lehrveranst
)
```

Das Schlüsselwort *references* bedeutet, dass auf ein Attribut (*foreign key ... references* den Primärschlüssel) einer anderen Relation gezeigt wird und in dieser nicht ohne weiteres die

Elemente entfernt werden können. Gleichzeitig kann aber kein Element in der Relation *Hält* existieren, das kein passendes Paar in der referenzierten Relation besitzt.

Zum Schlüsselwort *cascade* gibt es die Versionen *on update* und *on delete*. Das bedeutet, wenn sich etwas an den Attributausprägungen in der referenzierten Relation ändert oder gelöscht wird, überträgt sich das auch auf die Ausprägung dieser Relation.

4.4 Die Relationale Algebra [F16T1T1A2]

4.4.1 Grundoperationen

Die Grundoperationen der relationalen Algebra sind:

Vereinigung	$R = S \cup T$
Differenz	$R = S - T$
Kartesisches Produkt	$R = S \times T$
Selektion	$R = \sigma_F(S)$
Projektion	$R = \pi_{A,B,\dots}(S)$

Dabei ist F ein logisches Prädikat (z.B. $\text{Abteilungsnummer}=01 \wedge \text{Name}=\text{Mayer}$). Die Selektion nach einem kartesischen Produkt ist ein **Join**.

4.4.2 JOINS

Es gibt drei Arten von Joins:

Theta-Join	$R = S \bowtie_{A\Theta B} T$	Θ ist eine Operation (logisch, arithmetisch), A und B sind Attribute.
Equi-Join	$R = S \bowtie_{A=B} T$	Ausprägungen werden verbunden, wenn sie in den Attributen A und B gleich sind. Diese Information fehlt dann in der neuen Relation.
Natural-Join	$R = S \bowtie T$	Die Relationen werden auf allen gleichnamigen Attributen verbunden.

Ist der
Thetha-
Join
Staats-
examens-
relevant?

Ein Beispiel dafür ist:

$$R = \pi_{\text{SName}} \left(\text{Städte} \bowtie_{\text{LName}=\text{Land}} \sigma_{\text{Partei}=\text{CDU}}(\text{Länder}) \right) \quad (\text{Relationenkalkül})$$

Es werden alle Städte aus den Ländern ausgegeben, die von der CDU (mit)regiert werden.

Im Kontext von SQL-Anfragen gehen wir später noch auf typische Variationen wie inner/outer join ein.

4.4.3 Divisionen

Der Divisionsoperator ist eine Spezialität der relationalen Algebra. Er kann in SQL nur aufwändig mit vielen Joins und anderen weiteren Statements reproduziert werden.

Die Aussage $R_1 \div R_2$ bedeutet, dass in der Relation 1 alle Tupel ausgegeben werden, die in den übereinstimmenden Attributen mit der rechten Seite alle Einträge der rechten Relation als Teilmenge haben. Das bedeutet, dass in einer Liste mit Programmierern und

ihrer Programmiersprache nur diejenigen Namen ausgegeben werden, die alle Programmiersprachen eingetragen haben, die in der rechten Seite vermerkt sind. (In der damaligen Klausur: Alle Gruppen, die an jedem Hackathon in Hamburg teilgenommen haben)

Eine Möglichkeit der Umsetzung mit korrelierter Unterabfrage:

```
SELECT * FROM suppliers AS s
WHERE NOT EXISTS (
  ( SELECT p.pid FROM parts AS p )
  EXCEPT\
  (SELECT sp.pid FROM supplies sp WHERE sp.sid = s.sid )
);
```

Die Query funktioniert wie folgt: Es werden einzeln die Lieferanten durchgegangen. Es werden dann die ausgegeben, bei denen die Subquery leer bleibt. In dieser wird aus der Liste jedes Teil aussortiert, das der aktuell betrachtet Lieferant auch bringt. Ist die Unteranfrage leer, so liefert er alle Teile.

4.4.4 SQL [H96A4, F15T1T2A1, F19T1T1A2, ...]

SELECT

FROM

```
SELECT *
FROM Mitarbeiter m, Raeume r
```

WHERE

Die Einträge nach dem **WHERE** enthalten logische Operationen, diese können arithmetische Ausdrücke erhalten. So ist es möglich, Werte aus Feldern noch zu manipulieren um sie dann bspw. zu vergleichen.

Eine Besonderheit ist **LIKE**. Hier wird ein inexakter Vergleich gemacht. Hierbei ist das Prozentzeichen dann ein Platzhalter.

```
SELECT *
FROM Colour m, Raeume r
WHERE r.nr LIKE '0%' AND r.sitze => 50
```

Also alle Räume die mit 0 beginnen (im Erdgeschoss sind) und mindestens 50 Sitzplätze haben.

JOIN

Ein JOIN wird als Equals-Join realisiert, d.h. dass in der WHERE zwei Attribute miteinander in Gleichheit gebracht werden. Sinnvoll ist das vor allem auf Freundschlüsseln.

In neueren SQL-Dialekten sind auch die folgenden Joins erlaubt und äquivalent:

```
A JOIN B ON A.x = B.x
A JOIN B USING x
A NATURAL JOIN B    // Alle gleichnamigen Attribute werden kombiniert
```

Folgendes Beispiel: Nur Informationen zu Räumen und Mitarbeitern werden ausgegeben, die auch von Jan belegt sind.

```
SELECT *  
FROM Mitarbeiter m JOIN Raeume r ON r.belegung = m.name  
WHERE m.name = 'Jan'
```

JOINS sind nicht immer verlustfrei. Wenn also ein Datensatz aus der linken oder rechten Seite nicht passend gejoint werden kann, fällt er aus dem Ergebnis. Das kann vermieden werden durch LEFT JOIN, RIGHT JOIN und FULL JOIN. Diese sind dann links-/rechtsseitig verlustfrei.

Ein einfaches [INNER] JOIN ist immer auf beiden Seiten mit Verlusten möglich.

Verbindungen

Es können auch mehrere Anfragen (`SELECT ... FROM ... WHERE ...`) miteinander verbunden werden. Beispielsweise kann damit eine Untermenge von einer Obermenge abgezogen werden.

UNION	Verbindung mit Duplikatelimination
UNION ALL	Verbindung ohne Duplikatelimination
INTERSECT	Schnittmenge
EXCEPT	Subtraktion der zweiten von der ersten Relation
UNION CORRESPONDING	Es werden nur gleiche Attribute übernommen. Dabei kann sich in der zweiten Relation die Reihenfolge ändern

Subquery

Wie später beim Tupel- und Bereichskalkül kann in den bisher gelernten Möglichkeiten ein Quantor nicht explizit verwendet sondern nur im Relationenkalkül simuliert werden. Dafür gibt es die Subquerys. Diese werden zuerst ausgewertet und dann wird damit die Hauptanfrage verfeinert.

Eine Subquery wird in der `WHERE` Klausel verwendet und wird mit `EXISTS` aufgerufen. Alternativ zur Simulation eines Allquantors `WHERE NOT EXISTS` und dann eine negation der Formel im Allquantor. Das Beispiel mit den SPD-Alleinregierungen:

```
SELECT *  
FROM Länder L1  
WHERE NOT EXISTS (  
  SELECT *  
  FROM Länder L2  
  WHERE L1.LName = L2.LName AND NOT L2.Partei = "SPD"  
)
```

An allen Stellen einer SQL-Query, bei der eine Konstante steht, kann auch eine Subquery eingefügt werden. Bei dieser ist dann **nur ein Tupel und ein Attribut** erlaubt (damit wieder eine Konstante rauskommt). Das nennt man eine **direkte Subquery**.

Es kann auch für einen Vergleich eine Subquery aufgebaut werden. Dafür wird hinter den Vergleichsoperator dann ein `ALL`, `ANY` oder `SOME` geschrieben und dann eine Subquery.

Wie wird das explizit gemacht und ist es Stexrelevant??

In dieser ist dann logischerweise wieder nur ein Attribut erlaubt, hingegen aber mehrere Tupel. Auch damit können dann wieder Quantoren simuliert werden.

Ebenfalls kann ein **IN** und eine Subquery verwendet werden. Das ist äquivalent zu **ANY**.

Zueinander äquivalente Querys:

```
SELECT *
FROM MagicNumbers
WHERE NOT EXISTS (
    SELECT Zahl
    FROM Primzahlen
    WHERE Wert = Zahl
)

SELECT *
FROM MagicNumbers
NOT IN (SELECT Zahl FROM Primzahlen)

SELECT *
FROM Primzahlen
WHERE Wert <> ALL (SELECT Zahl FROM Primzahlen)
```

Korrelierte und unkorrelierte Unteranfragen

Eine **korrelierte Unteranfrage** ist eine solche, bei der aus der Hauptanfrage einer oder mehrere Werte berücksichtigt werden müssen. Ein Beispiel ist dabei die Abfrage aus dem Kapitel ???. Das bedeutet insbesondere, dass diese Abfrage für jedes Ergebnistupel erneut durchgeführt werden muss.

Anders hingegen verhält sich eine **unkorrelierte Unterabfrage**: Diese muss nur ein einziges Mal ausgewertet werden und ist daher deutlich performanter, aber nicht immer geeigneter. Ein Beispiel ist die zweite Abfrage der drei äquivalenten Anfragen aus diesem Unterkapitel. Beide Typen der Unterabfrage kommen meistens in der **WHERE** Klausel vor, sind aber auch woanders möglich.

Gruppierungen

Sind in einer Relation in einem Attribut gleiche Einträge vorhanden, so können diese gruppiert werden. Die weiteren Projektionen müssen dann Funktionen sein, die die weiteren Attribute gruppiert verarbeiten können (z.B. **count(*)** oder **SUM()**). Soll nach diesen neuen Attributen gefiltert werden, kann **HAVING** verwendet werden. (z.B. **HAVING SUM(salary) > 50.000**)

Manipulation der Relationen

Manipulation der Tupel

Mithilfe der Befehle **UPDATE**, **INSERT** und **DELETE** können die Tupel in den Relationen verändert werden.

Abschnitt
für
CREATE
TABLE,
DROP
TABLE,
ALTER
TABLE
ADD und
ALTER
TABLE
DROP
COLUMN
hin-

Das Update kann auf einer Relation durchgeführt werden. Dabei können mehrere Attribute manipuliert werden und dabei andere mit einbezogen werden. Außerdem können die Tupel eingeschränkt werden durch Bedingungen.

```
UPDATE Angestellte
SET Gehalt = Gehalt * 1,02
WHERE Gehalt < 3000
```

Das Entfernen löscht ein ganzes Tupel, das die Bedingungen erfüllt. Sind keine angegeben, so wird die gesamte Relation gelöscht. Dabei kann es zu Kaskaden kommen.

```
DELETE FROM Angestellte
WHERE Gehalt = 0
```

Die neuen Werte werden so eingefügt wie angegeben. Sind dabei nicht alle Attribute benannt, so werden die restlichen Werte mit **NULL** gefüllt. Ist das nicht möglich, so wird ein Fehler vom DBMS geworfen.

```
INSERT INTO Angestellte (Name,PNr)
VALUES ('Peter',26)
```

Ich denke nicht, dass die ganze Relation gelöscht werden kann.

VIEW

4.4.5 Tupel-Kalkül [F13T1T2A2, F20T1T2A3, ...]

Beim Tupel-Kalkül gibt es Atome und Formeln. Diese sollen dann zu Ausdrücken zusammengestellt werden. Ganz wie in der Mathematik:

$$\{t \mid \Psi(t)\}$$

Alle Tupel in der Relation, die die Formel Ψ wahr machen. Ein Beispielausdruck kann dann sein:

$$(\forall s \in R)(s.A < u.B \wedge u.C = t.D)$$

Dabei ist s eine gebundene Variable, t bleibt frei und u wird beim zweiten Aufruf gebunden. Beispiele dafür sind:

$$\begin{aligned} \text{Schema}(t) &= \text{Schema}(\text{Städte}) \{t \mid \exists(u \in \text{Länder}) \\ &\quad (u.LName = t.Land \wedge u.Partei = CDU)\} \\ \text{Schema}(t) &= \text{Schema}(\text{Länder}) \{t \mid \forall(u \in \text{Länder}) \\ &\quad (u.LName = t.LName \implies u.Partei = SPD)\} \end{aligned}$$

Hier sieht man auch die Feinheit der Quantoren: In der ersten Beschreibung werden alle Städte gefunden, die von der CDU (mit-)regiert werden. Die zweite Beschreibung steht für alle Länder, die von der SPD alleine regiert werden.

Erstellung von Views (Wurde mal abgefragt)

Ich verstehe das Beispiel nicht

4.4.6 Bereichskalkül

Im Gegensatz zum Tupelkalkül werden hier nicht die ganzen Tupel angefragt, sondern es kann nur mit den Variablen der Relation gerechnet werden. Dabei sind diese Variable oder auch fest belegbar. Zusätzlich können wieder auch Formeln und logische Prädikate verwendet werden.

$$\{x_1, \dots, x_l \mid \Psi(x_1, \dots, x_n)\} \quad l \leq n$$

Also können damit gezielt Informationen gefiltert werden (wie bei einem **SELECT**). Die beiden Beispielanfragen wie bereits zuvor in den anderen Kalkülen:

$$\begin{aligned} &\{x_1 \mid \exists x_2, x_3, y_1 (\text{Städte}(x_1, x_2, x_3) \wedge \text{Länder}(x_3, y_1, \text{CDU}))\} \\ &\{x_1 \mid \exists x_2 (\text{Länder}(x_1, x_2, \text{SPD})) \wedge \neg \exists y_3 (\text{Länder}(x_1, x_2, y_3) \wedge y_3 \neq \text{SPD})\} \end{aligned}$$

4.5 Entity-Relationship-Modell [F13T1T2A1, F15T1T2A3, ...]

Das Modell ist eine hohe Abstraktion eines Sachverhalts aus der realen Welt. Dabei ist es das Ziel, dieses Modell dann Maschinennah zu transformieren, um es in eine geeignete Datenbank zu überführen. Die gezeigte Notation entspricht der Chen-Notation. Der einzige Unterschied, der noch fehlt, ist, dass es zusammengesetzte Attribute gibt (male Attribute zu Attributen (z.B. Vorname und Nachname zu Name)) und es gibt abgeleitete Attribute. Diese sind in gestrichelten Ovalen gesetzt.

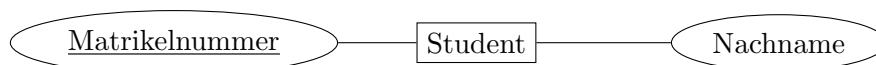
4.5.1 Elemente des E-R-Modells

Es werden drei Elemente unterschieden:

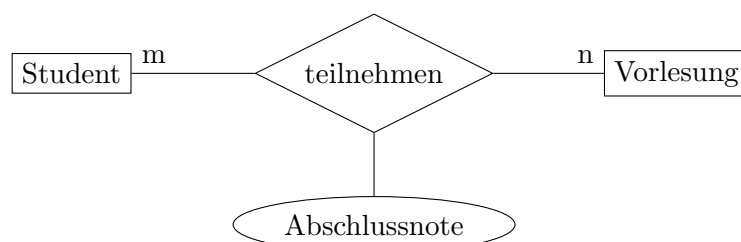
- **Entities:** sind Dinge die auch in der echten Welt existieren. Sie werden hauptsächlich durch Nomen signalisiert. Sie werden in Rechtecken dargestellt.

Student

- **Attribute:** sind Beschreibungen der Entities. Dabei sollen diese durch primitive Datentypen ausdrückbar sein (Int, Char, etc...). Attribute werden in runde Blasen gesetzt. Bei Primärschlüsseln wird zusätzlich der Name unterstrichen.



- **Relationships/Beziehungen:** sind die Beziehungen von Entities zueinander oder zu sich selbst. Die Darstellung erfolgt durch Rauten. An den Enden der Verbindungen werden gemäß Abschnitt ?? Kardinalitäten angegeben. Relationships können Attribute haben oder zwischen mehreren Entities sitzen.



Ebenfalls ist eine Beziehung von einer Entity zu sich selber möglich wie im Beispiel: n Veranstaltungen sind Voraussetzung für m andere.

Die Abbildung ?? zeigt ein typisches E/R-Diagramm. Allerdings sind hier keine Primärschlüssel eingetragen.

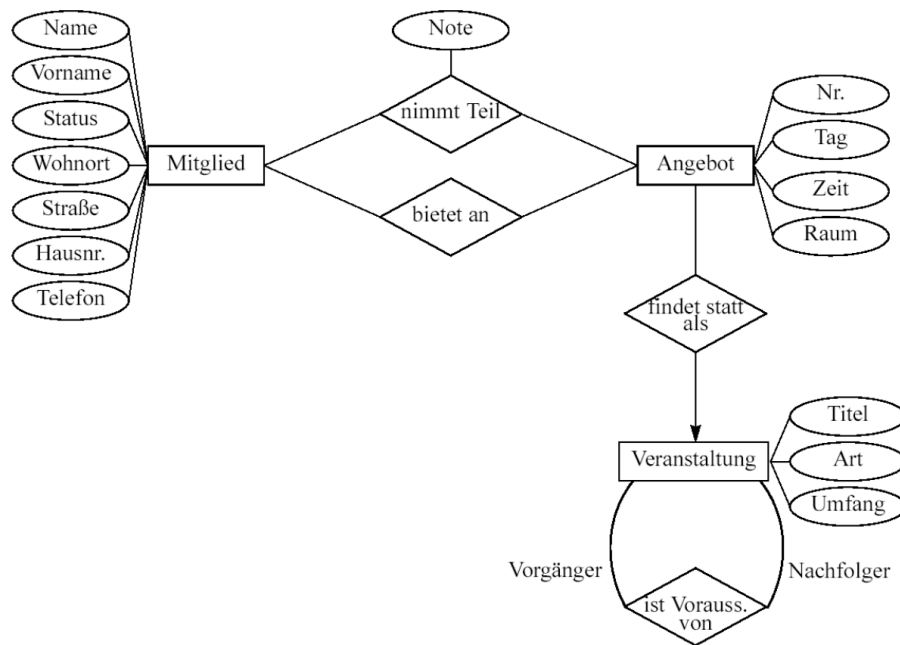


Abbildung 4.3: Beispiel eines E/R-Diagramms

4.5.2 Funktionalität von Relationen

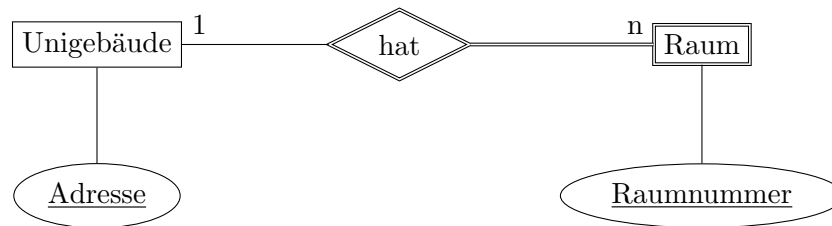
Um die Beziehung besser auszudrücken gibt es verschiedene Funktionalitäten und deren Darstellung:

- **1-zu-1 Beziehung:** Hier stehen die Entities in einer genauen Beziehung zueinander. Insbesondere gehört die eine Entity nur zu der anderen. Die Darstellung erfolgt mit einem Pfeil in beide Richtungen oder mit den Zahlen 1 und 1 an beiden Enden.
- **1-zu-n Beziehung:** Hier steht eine Entity zu vielen anderen in Beziehung. Sie werden als Pfeil in Richtung der einzelnen Entity oder den Bezeichnungen 1 und n dargestellt. In dem Beispielbild heißt es also, dass n Angebote als 1 Veranstaltung stattfinden.
- **n-zu-m Beziehung:** Hier stehen beliebig viele Entities auf beiden Seiten zueinander in Beziehung. Hierbei wird einfach nur ein Strich gezogen ohne Pfeilspitze oder n und m an den Enden vermerkt. Im Beispiel: n Mitglieder nehmen an m Angeboten teil.

4.5.3 Schwache Entitäten

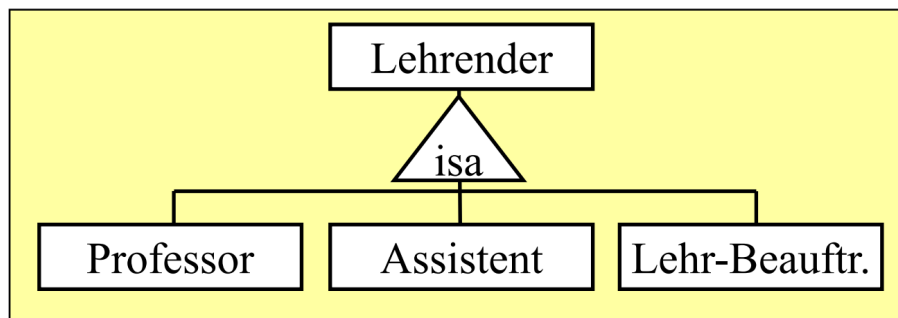
Eine schwache Entität ist eine solche, die ohne die Beziehung zu einer anderen Entität gar nicht existieren könnte. Diese werden dann doppelt umrahmt und doppelt verbunden zu

der Relation, diese ist ebenfalls doppelt umrahmt. Das Beispiel könnte eine Raumnummer sein, die nur mit der Angabe eines Universitätsgebäude eindeutig wird.



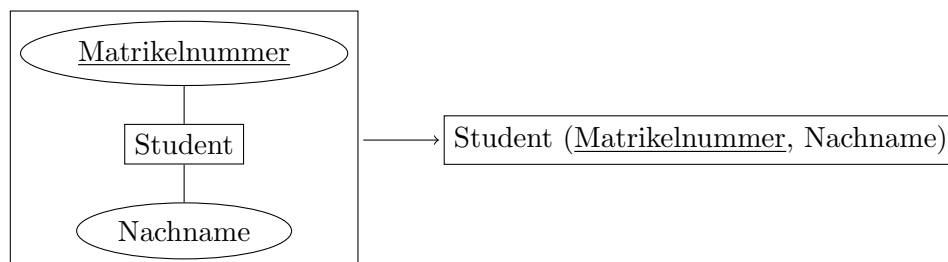
4.5.4 Vererbung in E-R

Die Vererbung in dem E-R-Modell wird durch ein Dreieck mit der Bezeichnung *ISA* angezeigt. Das bedeutet, dass alle weiteren Entities die Attribute erben, also die gleiche Ausprägung haben.



4.5.5 Übersetzung E-R zum relationalen Modell

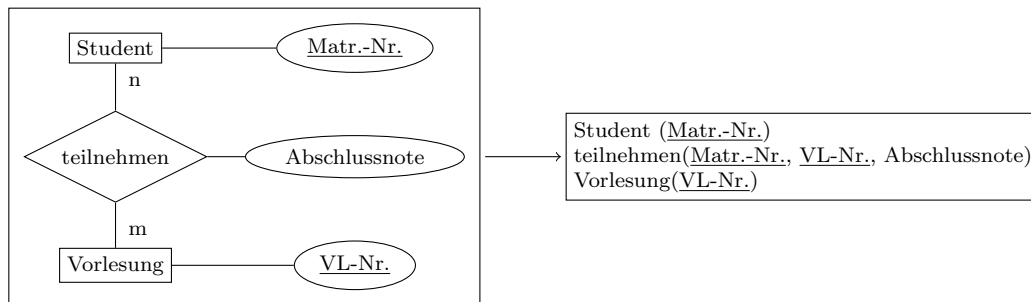
Eine typische Aufgabe des Staatsexamens ist es, das E-R-Modell in ein relationales Modell zu überführen. Grundsätzlich werden dazu einfach Entities mit ihren Attributen in Relationen übersetzt, wobei der Primärschlüssel einfach übernommen wird.



Relationships übersetzen

- Bei einer *one-to-many* Beziehung wird das Primärattribut aus der *one* Seite in die *many* Seite als Fremdschlüssel geschrieben. Die restlichen Attribute bleiben erhalten. Attribute der Beziehung werden ebenfalls in die *many* Relation geschrieben.
- Bei einer *many-to-many* Beziehung wird die Beziehung als neue Relation angelegt. Die Primärschlüssel beider Seiten werden Fremdschlüssel der neuen Relation und bil-

den zusammen den neuen Primärschlüssel. Weitere Attribute der Beziehung werden als Attribute in die neue Relation eingefügt.



- Bei einer *one-to-one* Beziehung wird wie bei einer *one-to-many* Beziehung gearbeitet. Der Unterschied ist, dass ein Entity komplett aufgelöst wird und in die Relation des anderen geschrieben wird. Dabei wird einer der vorherigen Primärschlüssel der neue Primärschlüssel. Durch die Eindeutigkeit der Schlüssel und der 1:1 Beziehung bleibt auch der neue Primärschlüssel eindeutig.
- Bei mehrstelligen Beziehungen kann verfahren werden wie bei den anderen Beziehungsarten. Bei einem Selbstbezug müssen wieder die Stelligkeit beachtet werden und wie zuvor beschrieben verfahren werden.
- Bei einer *ISA* Beziehung wird für jeden Erben verfahren wie in einer *one-to-many* Beziehung.

4.6 Normalformen

4.6.1 Funktionale Abhängigkeit und Anomalien [H02T2A2, F20T1T2A2, ...]

Motivation: In einer Relation können viele Doppelungen auftreten. Das heißt, dass beispielsweise einem Lieferanten mehrere Produkte in unterschiedlichen Tupeln zugeordnet wird. Die Redundanzen treten dadurch auf, dass die Stadt und das Land des Lieferanten jedes Mal eingetragen wird. In einem solchen Szenario wird die Abhängigkeit **Funktionale Abhängigkeit** genannt. Da aus der gleichen Stadt auch immer das gleiche Land folgt.

In solchen schlecht entworfenen Relationen können folgende Anomalien auftreten:

- **Update-Anomalie:** Beim Erneuern eines Eintrags (bspw. Adresse) wird nur eines der Tupel verändert. Das heißt, dass alle anderen Tupeln desselben Lieferanten dann falsche Adressen stehen.
- **Insert-Anomalie:** Wird ein neuer Lieferant angelegt, so muss auch immer direkt ein Produkt geliefert werden. Es ist nicht möglich, einfach nur einen Kontakt anzulegen.
- **Delete-Anomalie:** Wird der letzte Eintrag einer Lieferung gelöscht, so wird auch der Lieferant komplett gelöscht. Es ist also nicht möglich, den Lieferant für später abgespeichert zu halten.

Die Idee ist es also, alle funktionalen Abhängigkeiten in eigene Relationen aufzulösen.

Den ganzen Abschnitt müsste man neu strukturieren. Und die Armstrong-Axiome sind bereits abgefragt worden. Die fehlen hier

Definition: Funktionale Abhängigkeit

Sind X, Y Mengen von Attributen einer Relation R . Dann ist äquivalent:

- Y ist von X funktional abhängig (geschrieben: $X \rightarrow Y$)
- $\forall t, r \in R : t.X = r.X \implies t.Y = r.Y$

Attribute die zu einem Schlüsselkandidaten gehören werden **prim** genannt.

Definition: Volle und partielle Abhängigkeit

Sind X, Y Mengen von Attributen einer Relation R . Dann gilt:

- Y ist von X voll funktional abhängig (geschrieben: $X \twoheadrightarrow Y$), wenn es keine Teilmenge $X' \subsetneq X$ gibt sodass $X' \rightarrow Y$ gilt.
- Andernfalls heißt die Abhängigkeit partiell.

4.6.2 Attributhülle und Normalform [F21T1T2A4, F21T2T2A6, ...]

In der Attributhülle sind alle Attribute gespeichert, von denen es eine funktionale Abhängigkeit zu deren Erzeuger gibt.

Dazu füge alle direkten Abhängigkeiten vom Erzeuger der Hülle hinzu und wiederhole das so lange, bis keine weiteren Abhängigkeiten mehr genutzt werden können.

1. Normalform

Jede relationale Datenbank, jedes relationale Schema ist immer in der 1. Normalform. Dort müssen alle Attribute atomar sein.

2. Normalform

Ein Schema ist in zweiter Normalform, wenn gilt:

- Ein Attribut ist entweder voll funktional abhängig von allen Schlüsselkandidaten oder
- es ist prim.

Somit kann die 2. Normalform nur verletzt sein, wenn ein mehrstelliger Schlüsselkandidat existiert oder wenn es nicht-prime Attribute gibt.

Das Umwandlungsschema von 1. in zweiter Normalform läuft wie folgt: Für jede partielle Abhängigkeit wird die echte Teilmenge X_i des Schlüssels betrachtet, sodass $X_i \rightarrow Y_i$ gilt.

1. Die Tupel mit gleichem X_i werden gruppiert und zusammengeführt.
2. Die Menge Y_i wird entfernt und in eine neue Relation geschrieben.
3. X_i wird der Schlüssel der neuen Relation und verweist auf die alte Relation.

3. Normalform

Ein Schema ist in 3. Normalform, wenn gilt:

- Für jede nicht-triviale Abhängigkeit $X \rightarrow Y$ ist X Teil eines Schlüssel(-kandidaten) oder
- Y ist prim.

Der Umformungsalgorithmus bleibt gleich wie bei der Transition von 1. in 2. Normalform. Die Unterschiede sind:

- Attribute bleiben in der Relation, wenn sie voll funktional abhängig sind vom gesamten Schlüssel und nicht von einem nicht-primen Attribut abhängig sind.
- Es werden auch die Attribute gruppiert und entfernt, die von einem nicht-primen Attribut abhängen. Für diese wird ebenfalls eine neue Relation erzeugt.

Verlustlosigkeit und Abhängigkeitserhaltung [F94A7]

Eine Zerlegung ist **verlustlos**, wenn gilt:

$$R = R_1 \bowtie R_2$$

Eine Zerlegung ist **abhängigkeitserhaltend**, wenn alle funktionalen Abhängigkeiten in mindestens einer der Relationen vollständig enthalten ist, bzw. wenn die Attributhüllen der Abhängigkeiten gleich sind.

$$F_R^+ = \left(F_{R_1}^+ \cup F_{R_2}^+ \right)$$

Kanonische Überdeckung der funktionalen Abhängigkeiten [H15T1T1A2]

Hierbei geht es darum aus einer Liste der funktionalen Abhängigkeiten eine neue zu generieren, die möglichst kurz ist.

1. **Linksreduktion:** Prüfe für jede Abhängigkeit ob Teile der linken Seite überflüssig sind. Streiche diese entsprechend.
2. **Rechtsreduktion:** Prüfe, ob eine rechte Seite bereits durch andere (transitive) Abhängigkeiten gegeben ist. Streiche diese rechte Seite.
3. Entferne alle Abhängigkeiten mit leerer rechter Seite.
4. Fasse alle Abhängigkeiten zusammen, deren linke Seite exakt gleich ist.

Synthesealgorithmus für die 3. Normalform [F17T2T1A5, F19T1T1A3, ...]

Für eine gegebene Menge F von funktionalen Abhängigkeiten führe die folgenden Schritte aus:

1. Überführe die Menge F in die kanonische Überdeckung F_C .
2. Für jede Abhängigkeit $X \rightarrow A \in F_C$ füge eine neue Relation $R = X \cup A$ hinzu und ordne die verwandten Abhängigkeiten und alle Abhängigkeiten deren Linke und rechte Seiten in der neuen Relation enthalten sind der neuen Relation zu.
3. Rekonstruiere Schlüsselkandidaten: Wenn der Schlüsselkandidat bereits vollständig in einer dieser Relationen genutzt wird, muss nichts getan werden. Andernfalls: Erzeuge eine Relation, die den gesamten alten Schlüsselkandidaten enthält.

4. Eliminiere Relationen, die bereits vollständig in anderen enthalten sind.

Boyce-Codd-Normalform [H02T2A2]

Definition: Boyce-Codd-Normalform

Eine Relation R ist genau dann in BCNF, wenn für jede nichttriviale Abhängigkeit $X \rightarrow Y$ gilt:
 X enthält einen Schlüsselkandidaten von R .

Was hier gilt: Es gibt keine nichttrivialen Abhängigkeiten zwischen Schlüsselattributen.

Bewertung der Normalisierung

Bei einem klug designten System bedarf es eigentlich keiner oder nur wenig Normalisierung. Das Problem an Normalisierung ist, dass viele Joins ausgewertet werden müssen. Das ist kostenintensiv.

Ein Vorteil ist, dass es die Persistenz der Daten verbessert und den Speicherbedarf verringert.

Mit Integritätsbedingungen (z.B. Triggern) kann man die Anomalien vermeiden. Daher müssen bei der Normalisierung nicht immer alle funktionalen Abhängigkeiten berücksichtigt werden.

4.7 Anfragenoptimierung [H19T2T2A5]

Bei der Optimierung von Anfragen geht es darum, dass die Anfrage so umgestaltet wird, dass die Auswertung des Ausdrucks möglichst leicht/schnell/kostengünstig erfolgen kann. Dabei werden drei (im Staatsexamen anscheinend zwei) Arten unterschieden:

- **Logische Optimierung:** Diese Techniken beruhen darauf, die Abfolge der Operationen zu verändern.
- **Physische Optimierung:** Hierbei geht es um die Auswahl der tatsächlichen Algorithmen zur Auswertung/Durchführung einer Operation.
- **Regel- und kostenbasierte Optimierung:** Hierbei wird entweder nach einem festen Regelwerk optimiert oder nach verschiedenen Heuristiken, die es ermöglichen einen kostengünstigen Auswertungsplan zu wählen.

Logische Optimierung

Auf der relationalen Algebra lassen sich einige Äquivalenzregeln betrachten:

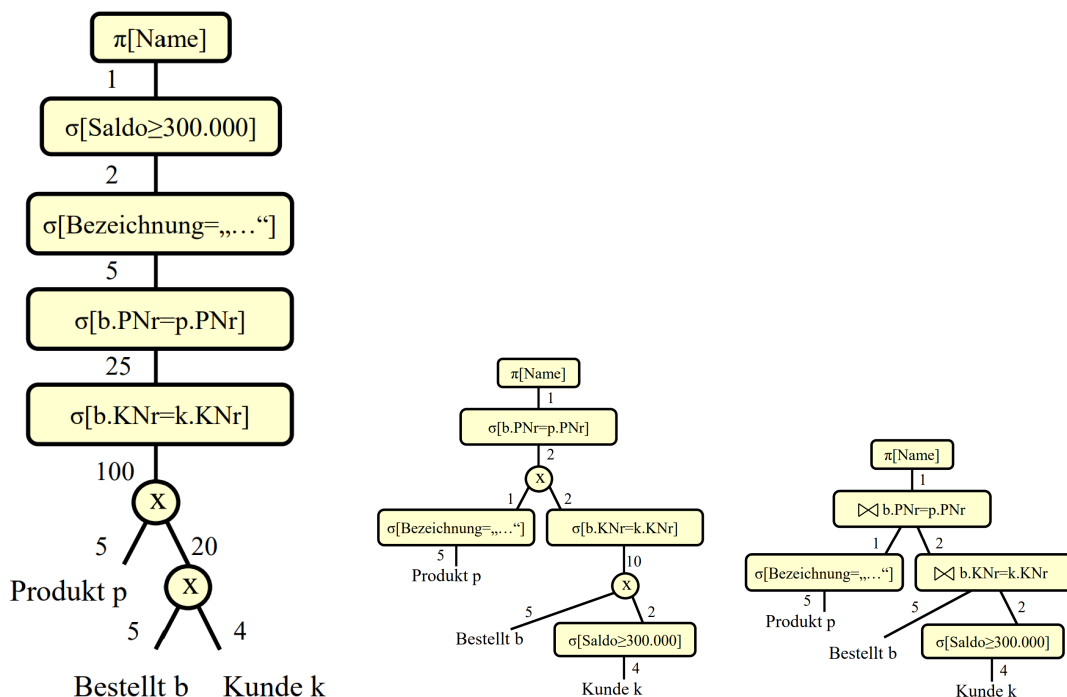
- Join, Vereinigung, Schnitt und Kreuzprodukt sind assoziativ und kommutativ.
- Selektionen sind vertauschbar.
- Selektionen mit logischen Operatoren können aufgebrochen werden zu einzelnen Selektionen.

- Projektionen können vernachlässigt werden, wenn sie geschachtelt sind und immer weniger nur noch betrachtet wird nach außen hin.
- Selektion und Projektion sind vertauschbar, solange die Projektion nichts mit der Selektionsbedingung zu tun hat.
- Selektionen nach Produkten können zu Joins gemacht werden.
- Selektionen können mit Joins verschoben werden unter Bedingungen.
- Selektionen können mit den anderen Mengenoperationen verschoben werden, Projektionen nur mit der Vereinigung.

Es gibt ein gutes Rezept zur logischen Optimierung:

1. Aufbrechen der Selektionen
2. Verschieben der Selektionen so weit wie möglich nach unten im Operatorbaum
3. Zusammenfassen von Selektionen und Kreuzprodukten zu Joins
4. Einfügen und Verschieben von Projektionen so weit wie möglich nach unten
5. Zusammenfassen einzelner Selektionen zu komplexen Selektionen

Darstellung der Optimierung im Operatorbaum



4.8 Transaktionen

4.8.1 ACID-Prinzip [F19T2T1A1, H19T2T2A6, ...]

Eine Transaktion folgt immer dem ACID-Prinzip. Das bedeutet:

- **Atomarität:** Eine Transaktion wird entweder ganz oder gar nicht ausgeführt.

- **Consistency:** Eine Transaktion überführt einen konsistenten Datenbankzustand in einen neuen konsistenten Zustand.
- **Isolation:** Eine Transaktion ist blind für die Änderungen anderer Nutzer zur gleichen Zeit. Also wird nur auf einem zu Beginn der Transaktion gültigen Zustand operiert.
- **Dauerhaftigkeit:** Der Effekt einer Transaktion ist dauerhaft (es kann aber durchaus die Möglichkeit zum Rollback geben).

4.8.2 Probleme der Synchronisation von Transaktionen [Ø]

Da die serielle Ausführung von Transaktionen nicht besonders effizient ist, muss das DBMS eine Möglichkeit der Synchronisation bieten. In diesem Mehrbenutzerbetrieb kann es dann zu den folgenden Anomalien kommen, die das DBMS verhindern muss:

Lost Updates

Zwei Transaktionen überschreiben das gleiche Objekt. Beide haben zuvor den gleichen Zustand gelesen aber nur eines der Updates bleibt erhalten. Das verstößt gegen das Prinzip der **Dauerhaftigkeit**.

Dirty Read und Dirty Write

Diese Anomalie entsteht, wenn eine Transaktion einen Wert ändert und eine weitere Transaktion diesen Zustand liest (dirty Read). Wird dann aber die Transaktion 1 zurückgesetzt und die zweite Transaktion schreibt trotzdem, dann ist das ein dirty Write.

Diese Vorgänge verstoßen gegen die **Dauerhaftigkeit** und die **Consistency**.

Non-repeatable Read

Eine Transaktion liest den Zustand der Datenbank zweimal aus. Währenddessen schreibt eine weitere Transaktion eine Änderung. Da die erste Transaktion noch nicht abgeschlossen ist, wird ein anderer Zustand gelesen.

Das verstößt gegen die **Isolation**.

Phantomproblem

Das Phantomproblem ist eine Variante des Non-repeatable Reads. Dabei fügt die zweite Datenbank ein neues Datum hinzu, während die erste Transaktion noch nicht abgeschlossen ist. Dadurch liest die erste Transaktion dann beim zweiten Read das Phantom mit ein.

4.8.3 Schedules [F21T1T2A5]

Es gibt zwei Arten von Schedules: Die **seriellen** und die **allgemeinen** Schedules.

- Ein **serieller** Schedule ist die Hintereinanderausführung von Transaktionen, bei dem die Aktionen nicht verzahnt sind. Diese sind aus Sicht der Isolation und Konsistenz gewünscht.

- Die **allgemeinen** Schedules sind solche, bei denen die Aktionen verschiedener Transaktionen verzahnt ablaufen. Das ist aus Sicht der Performanz gewünscht.

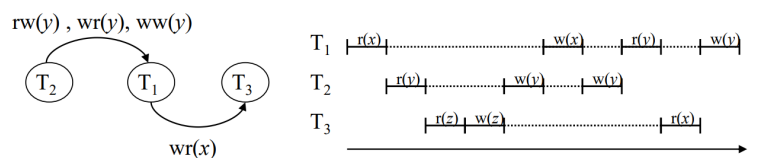
Es muss also der Mittelweg gefunden werden. In diesem Kontext existieren die **serialisierbaren Schedules**.

Beachte dafür folgende Notation:

- $r_i(x)$ bedeutet, dass Transaktion i auf x schreibt.
- $w_i(x)$ bedeutet, dass Transaktion i auf x liest.
- $rw_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x liest und dann j schreibt.
- $rw_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x liest und dann j schreibt.
- $ww_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x schreibt und dann j schreibt.

Ein Serialisierbarkeitsgraph hat alle Transaktionen als Knoten und die Abhängigkeiten (die letzte drei Punkte) werden als Kanten eingetragen. Ist der Graph zyklensfrei, so ist der Schedule serialisierbar. Ein möglicher Schedule ist dann eine Abfolge von Transaktionen,

Beispiel: $S = (r_1(x), r_2(y), r_3(z), w_3(z), w_2(y), w_1(x), w_2(y), r_1(y), r_3(x), w_1(y))$



die aus dem Graphen gelesen werden kann. Dabei kann eine Transaktion nur dann gewählt werden, wenn alle darauf zeigenden Transaktionen bereits durchgeführt wurden.

Um die Serialisierbarkeit zu gewährleisten, ist das 2-Phasen-Sperrprotokoll der Königsweg.

4.8.4 2-Phasen-Sperrprotokoll (2PL) [F21T1T2A1]

Das Protokoll verfolgt als Grundprinzip den folgenden Ansatz: Eine Transaktion fordert im Laufe der Zeit immer mehr Sperren an. Werden diese nicht mehr benötigt, so werden die Sperren zurückgegeben.

Möchte eine andere Transaktion auf einen gesperrten Teil (Relation, Attributwert, alles Mögliche denkbar) zugreifen, so muss sie warten.

Dieses Prinzip gewährleistet bereits Serialisierbarkeit. Sie verhindert aber keine Dirty Reads. Das passiert dann, wenn eine Transaktion ihre Sperre zurückgibt und daraufhin eine andere Transaktion auf den entsprechenden Daten liest. Produziert die erste Transaktion später einen Fehler, so muss nicht nur diese, sondern auch alle anderen Transaktionen mit Dirty Reads zurückgesetzt werden.

Daher gibt es das **strikte** 2PL. Hier werden die Sperren erst zurückgegeben, wenn die Transaktion erfolgreich beendet oder zurückgesetzt wurde.

In diesem Kontext fällt auf, dass es bei mehreren Lesern ohne Schreibern nicht zu einer Verletzung der Isolation kommt. Da das strikte 2PL sehr wenig Parallelität erlaubt, werden zwei Typen von Sperren im RX-Sperrverfahren verwendet:

- Lesesperren (R-Locks)
- Schreibsperrern (X-Locks)

Das impliziert, dass es möglich ist, auf einem Datum mehrere Lesesperren zu haben, da diese sich nicht gegenseitig behindern.

4.8.5 Integritätsbedingungen [H15T1T2A1]

Nach jeder Transaktionen, muss ein DBMS sicherstellen, dass bestimmte Integritätsbedingungen erfüllt bleiben. Beispiele hierfür sind:

- **Schlüssel-Integrität:** Schlüssel-Attribute bleiben eindeutig.
- **Referentielle Integrität:** Fremdschlüssel haben eine wohldefinierte Referenz.
- **Multiplizitäten Integrität:** Vorher festgelegte Multiplizitäten wir 1:m werden eingehalten.

4.9 Nicht behandelte Konzepte

4.9.1 Satzadressierung/Tupel-Identifikatoren (TIDs) [H16T1T1A5, F17T1T2A4]